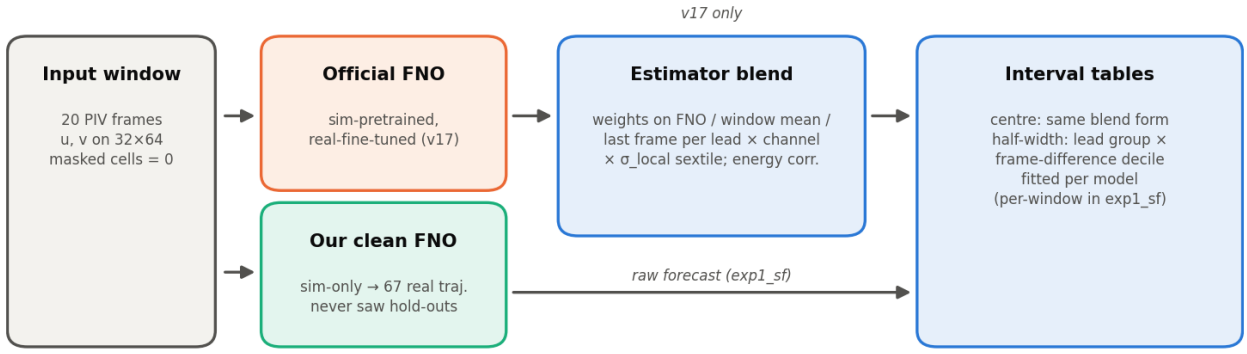
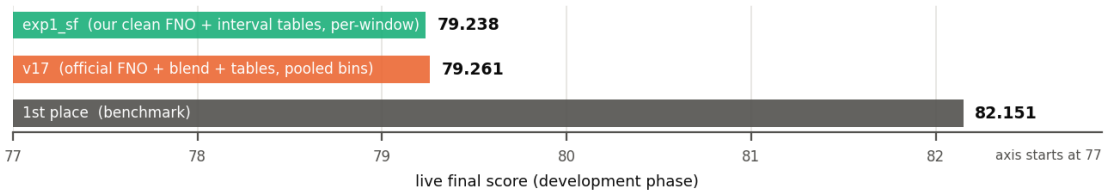
Input-Window Estimator Blending for Real PIV Forecasting under a Possibly Contaminated Local Hold-out

NeurIPS 2026 RealPDE Competition, Track 1 (Simulation-to-Real Transfer): technical report

Team **Slash** (Codabench: slash) · Shashwat Srivastava · Independent Researcher

79.238	**79.261**	**+3.95**	**82.151**	**45 · 19**
live score, our clean sim→real FNO (per-window intervals)	live score, v17: official FNO + our post-processing (pooled bins)	live SPS gain from interval placement (v17 vs v8)	1st place (benchmark)	experiments · live submissions





Overview. What we built: two forecasters, each with its own interval tables of the same form; v17 also blends the official FNO with the window mean and last frame. exp1_sf computes its statistics per window; the live v17 build pooled them across each call (Disclosure 1). Bottom: the two live results against the 1st-place benchmark (axis starts at 77).

Abstract. We forecast 20 future frames of real particle-image-velocimetry (PIV) velocity around a NACA4418 airfoil from a 20-frame input window.

Main results. We report two live numbers, each with a compliance qualification. (i) **79.238** (exp1_sf): our own sim-to-real fine-tune of the simulation-only FNO, which never saw a local hold-out, with per-window interval tables (independence test passed). Its tables were fitted on SELECT, a split chosen with logged evaluation-input statistics, so Disclosure 2 applies (§10, §13). (ii) **79.261** (v17, live, pooled build): a post-processing layer fitted on released training data, applied to the organisers' unchanged simulation-pretrained, real-fine-tuned Fourier Neural Operator (FNO), raised the live Safe Prediction Score (SPS) from 33.34 to 37.29 and the final score from 78.14 to 79.26. v17 pooled rank statistics across the windows of each call, which the Rules do not allow (Disclosure 1). Its per-window rebuild differs locally by +0.002 to +0.006 composite, measured against the pooled build recomputed with an approximation of the harness's grouping (five consecutive windows per call, each window's target being the next window's input; our groups can cross trajectory boundaries and are not padded); it was not scored live. The 1st-place score, 82.151, is our benchmark for the maximum achieved.

Why it works: statistics of the input window identify where persistence or the window mean is more accurate than the neural forecast, especially at short leads and in calm regions, where the FNO over-smooths. The same statistics place the prediction interval. Under a live-calibrated proxy, the interval-centre table alone adds 2.5 SPS points and the width table alone 1.8, of 4.0 in total; the attribution depends on the order in which components are added.

Sim-to-real: our fine-tunes of the simulation-only FNO raise the rel subscore on held-out real trajectories from 79.7 to 94.3. Retrained and submitted live (exp1_sf, 79.238), its raw forecast scored

93.73/77.35/92.38 against the official FNO's 94.17/74.03/92.84 on the same live set: it traded 0.44 rel and 0.46 mvpe for 3.32 tke. As full builds, exp1_sf and v17 are at composite parity (79.238 vs 79.261; §10).

Evaluation: the official checkpoint appears to have been trained on every released real trajectory. This is an inference, not documented; a clean control loses far less between FINAL and live (§10). If so, no local hold-out is unseen by it, and every local comparison favoured it. That is consistent with a record in which four locally validated changes delivered between −31% and +7% of their local effect live.

Record: we give an exact specification of the method (Algorithm 1), a component ablation, the decision rules we used, and the protocol of every record (45 experiments, 15 measurements, 19 submissions). We also disclose two compliance issues: rank statistics pooled across the windows of each call, and evaluation-input statistics written to logs by diagnostic submissions.

1 Task, metrics and what counts

Inputs are windows (N, 20, 32, 64, 3) with channels [u, v, p], and p ≡ 0 for PIV. Outputs have the same shape for 20 future frames, optionally with per-element *lower*/*upper* bounds. Five subscores are reported. Relative L2 (rel), turbulent kinetic energy (tke: relative L2 of the per-pixel temporal variance map) and mean velocity profile error (mvpe: four probe columns, time-averaged first) each map an error e to 100/(1+0.5e). A runtime score and the Safe Prediction Score complete the set:

$$\mathrm{SPS} = 100 \cdot \Sigma_b\, w_b \cdot \mathrm{mean}[(1-\mathrm{pm}_b) \cdot \exp(-(\mathrm{upper}-\mathrm{lower})/\sigma) \cdot 1\{\mathrm{covered}\}],\ \mathrm{pm} = e/(0.5+e),\ \sigma = 0.0563870,\ w = (0.5, 0.3, 0.2)\ \text{for (rel, tke, mvpe)}$$

SPS factorises closely as 100·W·T. W depends only on accuracy. T, the *band factor*, depends on how tight and well-placed the intervals are. The optimal coverage is 0.70–0.80, not 0.95, because over-wide bands are penalised exponentially. How the five subscores combine into final_score is unpublished.

2 Data audit and the evaluation set

Release defects. *train_real/6300_0* and *7575_0* are byte-identical. We reported this, and the organisers' data notice excluded *7575_0*. In 78 of 82 files the *re* attribute disagrees with the file name (the true Reynolds step is 1,278–1,279, not 1,275). *train_sim* matches the real grid, but its amplitude ratio is anisotropic (u_std 3.59×, v_std 10.81×) and it lacks PIV dropout (0.20% vs 9.91% zeros). We never used it for training.

The evaluation set. From aggregate input statistics our diagnostic submissions logged (see §13), the development evaluation set is a single flow condition, AoA 15°, Re ≈ 2.1–2.5×10^4. Its low-order input statistics (temporal autocorrelation, radial spectrum) are consistent with training trajectories of the same regime. We do not publish the logged values themselves while no organiser ruling exists (Disclosure 2). Live mvpe, below all 81 training trajectories (Fig. 2), is not explained by these statistics.

Full data, harness, training and testing details, with figures, are in Appendix D.

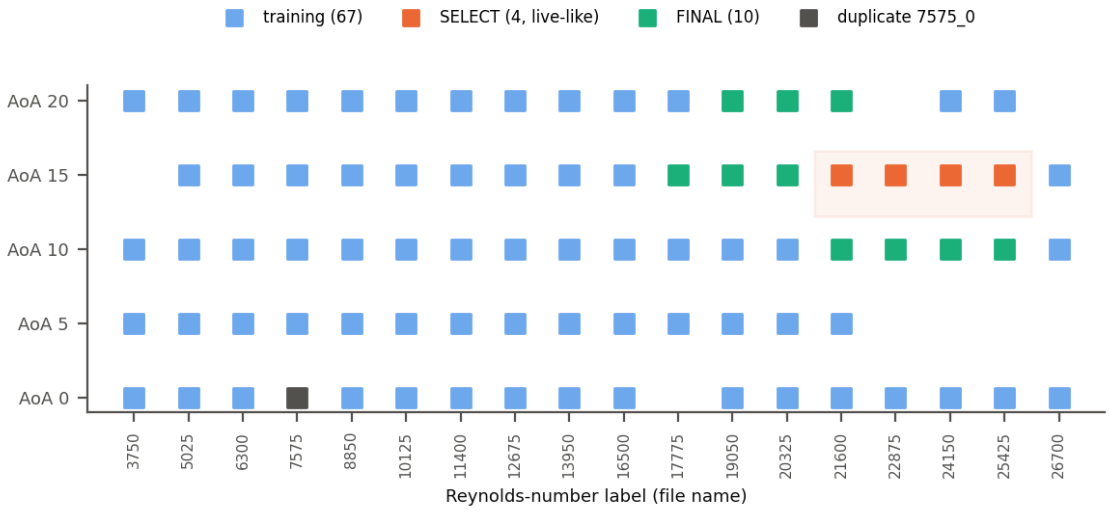


Figure 1. Our split of the 81 usable real trajectories: 67 for training, SELECT (4 AoA-15 trajectories at Re 21.6–25.4k, the regime of the live set, shaded) and FINAL (10). The duplicate 7575_0 is excluded.

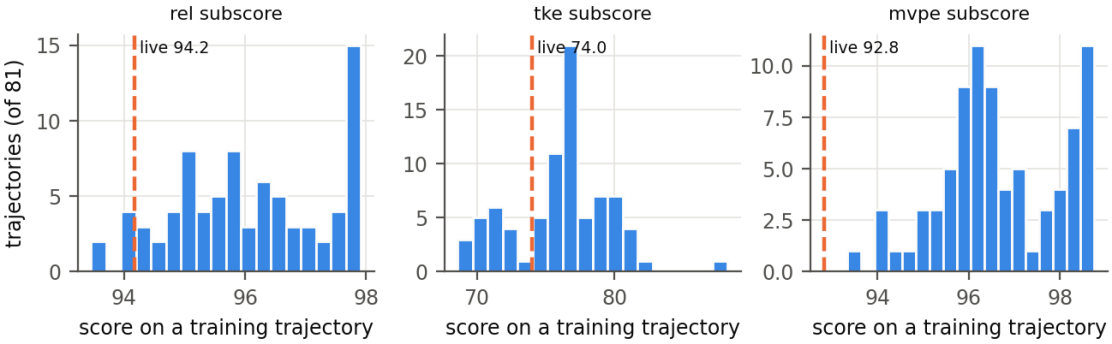


Figure 2. The official FNO scored on each of the 81 training trajectories (10 windows each), with its live development score (dashed). These trajectories were very likely in its training data (Table 2), so the spread need not reflect generalisation. Live mvpe lies below all 81.

3 Our live record and the benchmark

We made 19 live submissions (S1–S9, v7–v13, v15, v17, exp1_sf; Appendix A); nine have recorded subscores (S5 without a time score) and are plotted below. The only other-team benchmark in this report is the 1st-place score on the development board, 82.151 (rel 94.746, tke 80.191, mvpe 94.243, time 92.030, SPS 44.967), as the maximum achieved; the scoring sensitivities below are estimated from the public board as a whole. Our best is 79.261 (v17), 2.89 below it. Our best build without cross-window pooling is 79.238 (exp1_sf, §10).

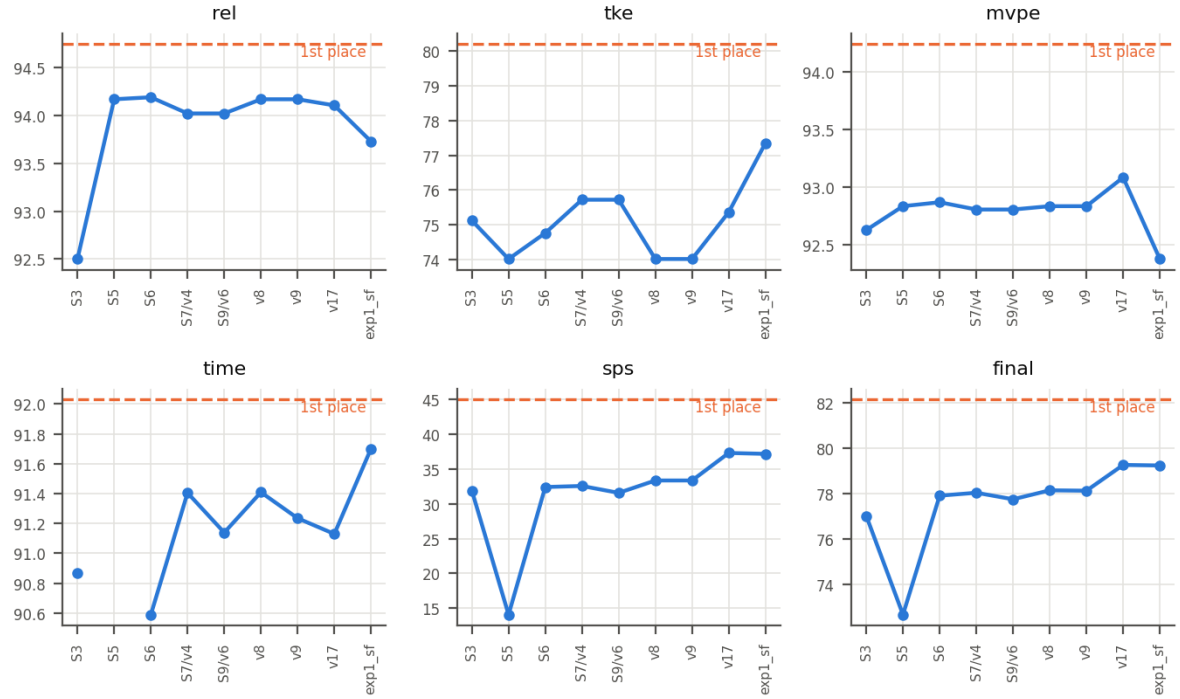


Figure 3. Every live submission with recorded subscores, per metric, against the 1st-place benchmark. S5 shipped without intervals (SPS 14.1). v17 and exp1_sf are the two final approaches: the official FNO with our post-processing, and our own clean sim-to-real FNO.

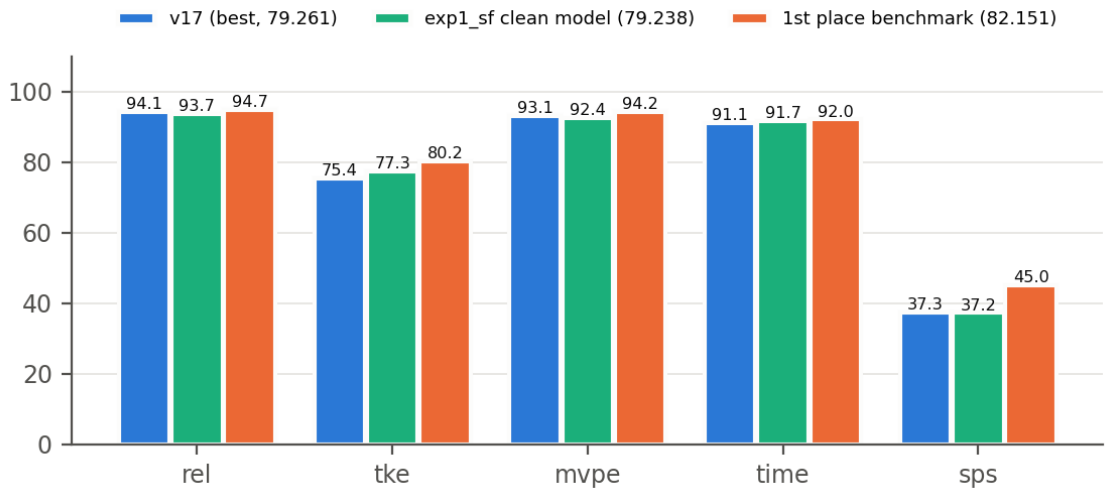


Figure 4. Our two final approaches against the 1st-place benchmark, per subscore. The largest distance is SPS (−7.7 for v17, −7.8 for the clean model) and tke (−2.8 for the clean model, −4.8 for v17).

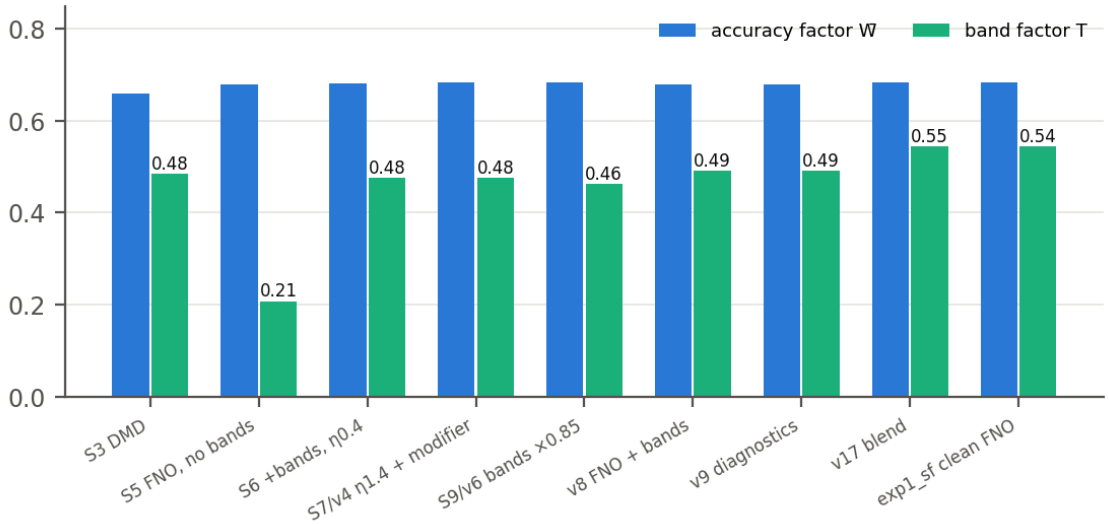


Figure 5. SPS of each live build split into the accuracy factor W and the band factor T (SPS ≈ 100·W·T). Accuracy barely moved across our builds; every SPS gain came from the intervals: default bands → heteroscedastic (T 0.21 → 0.48), then v17's centre and width tables (0.49 → 0.55).

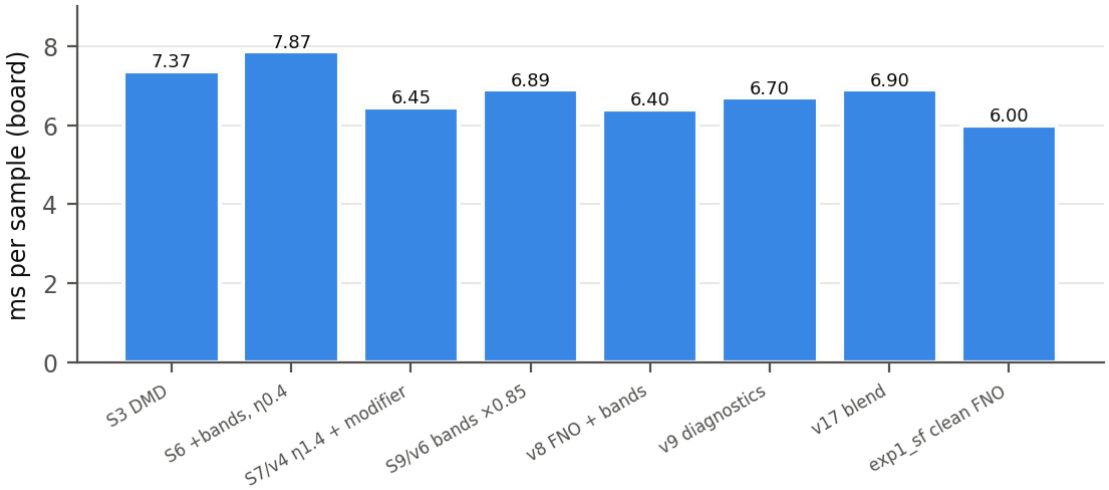


Figure 6. Measured runtime per sample on the evaluation server for each build.

Scoring sensitivities (method note). To rank candidates, we use a linear approximation of how final_score responds to each subscore, estimated from the public leaderboard: per subscore point rel 0.453, tke 0.087, mvpe 0.081, time 0.093, SPS 0.284 (bootstrap confidence intervals in

report/data/weights_bootstrap.json, in the deposit). Two of our own bit-identical builds that differed only in runtime give a directly measured time sensitivity of 0.095, and show that the board is deterministic for accuracy and SPS (only runtime varies, ±0.017 final).

4 Sim-to-real transfer experiments

Our first line of attack was to transfer the simulation-pretrained operators to real PIV ourselves (Table 1, Fig. 7). The clean-start experiments begin from checkpoints that never saw real data and train on 67 real trajectories. The 10 FINAL trajectories are held out, so these evaluations are honest. Full fine-tuning of the simulation-only FNO (Exp. 1) lifts rel/tke/mvpe on FINAL from 79.7/66.9/75.2 to 94.3/78.8/95.5. Freezing everything except the fc/BatchNorm layers (FT10) recovers most of the rel and mvpe gain with 0.02% of the parameters trainable.

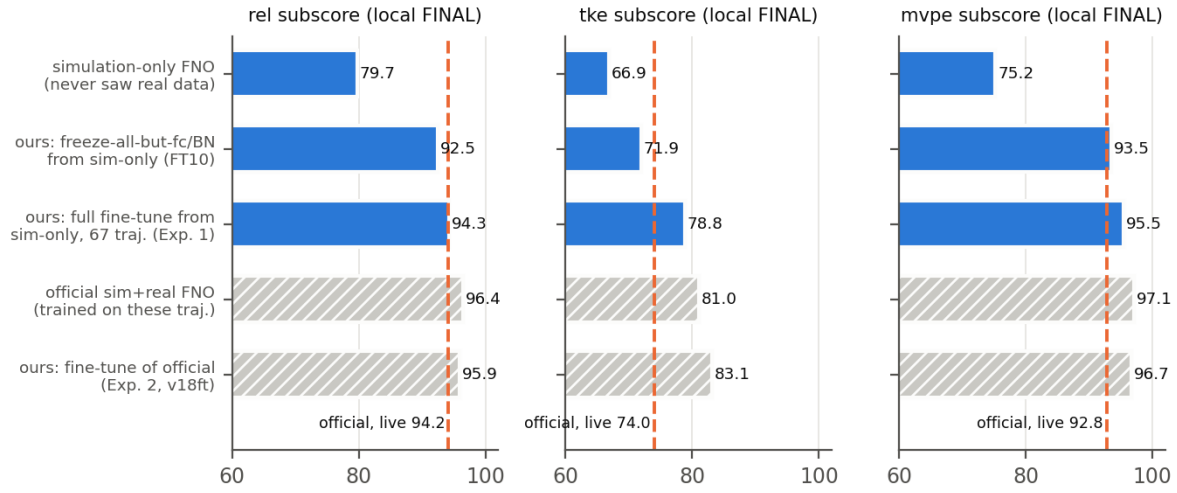


Figure 7. Sim-to-real transfer on the local FINAL split. Blue: models that never saw these trajectories. Grey hatched: the official sim+real checkpoint and our fine-tunes of it. The organisers' training set includes these trajectories (inferred; see §5), so those scores are optimistic. Dashed: the official model's live scores. The two are measured on different data and are not directly comparable.

experiment	start	trainable / data	FINAL rel / tke / mvpe	reading
Exp. 1 full fine-tune	sim-only FNO	all; 67 traj., TRUE_W loss	94.27 / 78.78 / 95.51	large honest sim→real gain; plateaued by epoch 10; never tested live
FT10 freeze-bn-fc + spectral aug.	sim-only FNO	fc/BN (0.02%)	92.46 / 71.94 / 93.53	wins all three vs its own start (+36 sum)
FT01 fine-tune, stride 20	official FNO	all; 3,362 windows	Δ −0.89 / +2.66 / −0.62	buys tke, sells rel (amplitude rescale)
FT02 9.3× windows, rel-heavy loss	official FNO	all; 31,343 windows	Δ −0.06 / −0.93 / −0.35	no productive direction once tke is forbidden
FT02 + amplitude jitter	official FNO	all	Δ −0.11 / −0.46 / −0.40	augmentation did not rescue it
Exp. 2/3 lr 2e-5 (v18ft)	official FNO	all; 67 traj.	Δ −0.48 / +2.10 / −0.40	same trade; memorised yardstick biased against it
FT11 freeze-bn-fc	official FNO	fc/BN	Δ −0.03 / −0.92 / −0.03	output calibration already optimal
WiSE-FT sim ↔ sim+real	both	interpolation	−7.5 composite at α=0.95	different basins (‖Δθ‖/‖θ‖ = 1.09)
CNO clean-start fine-tune	sim-only CNO	all	no result	out of memory on a 14.6 GB T4; never completed

Table 1. Fine-tuning experiments (register ids FT01–FT13, EM06). Δ rows are against the frozen official model on the memorised split. The rows starting from the sim-only model are honest held-out numbers.

Why the fine-tunes of the official model failed. The official checkpoint had already been fine-tuned on the same real data. From that converged point, the one direction that reduces the training loss is an output-amplitude rescale, which buys tke and sells rel. Forbid that trade and training overfits (run B: train loss −15% while held-out fell). And every one of these runs was judged against a checkpoint that had memorised the yardstick (§5). A fine-tune that generalises better could still look slightly worse locally. None was ever submitted live, so the question is open.

5 The evaluation trap and how we worked around it

We state the memorisation question once, here, with the strength of each claim. Elsewhere in the report, "memorised split" is shorthand for the plausible but unproven explanation in the third row.

claim	evidence level	basis
The official checkpoint was fine-tuned on real PIV data	documented	released as the sim+real fine-tuned baseline; its metadata records iteration, best_iteration and best_val_loss
It was trained on every released real trajectory, including our hold-out	inference	no statement that any trajectory was held out; release contains only the training split
Its local scores are inflated by having seen the local trajectories	supported by one clean control (n = 1)	it scores 96.38/81.04/97.06 on our FINAL split vs 94.17/74.03/92.84 live; our Exp. 1, which never saw FINAL, scores 94.27/78.78/95.51 there. FINAL→live drop: official raw FNO −2.21/−7.01/−4.23; clean Run 1 (never saw FINAL) −0.43/−1.83/−3.13 (§10). Alternative not fully excluded: the live flow sits in the regime cluster where the FNO is weakest
Local comparisons against it were biased in its favour	consistent with the record	four locally validated changes delivered −31% to +7% of their local effect live (Fig. 8); for v17, live SPS was 57–99% of the four proxies' predictions and live composite 54–84%

Table 2. The memorisation question, graded by evidence.

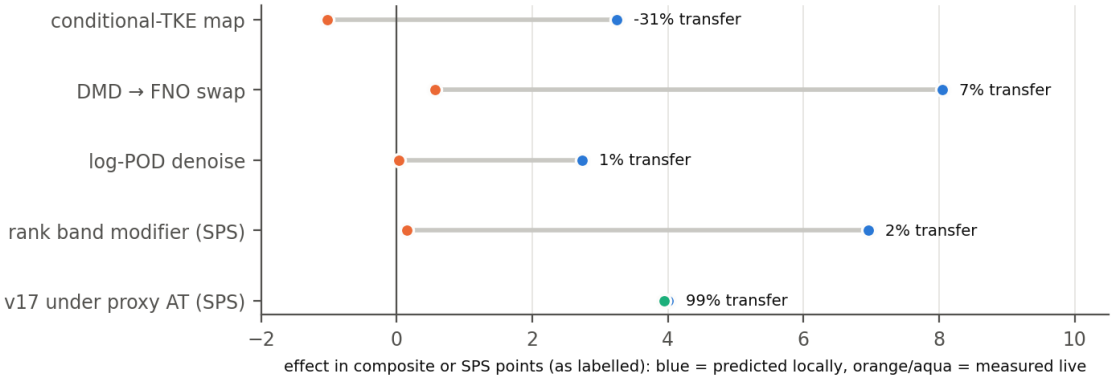


Figure 8. Local prediction vs live measurement for shipped changes. The first four were validated on the memorised split. The last was predicted before submission by one of four proxies (see Table 3), identified as the closest only after the live result was known. Units differ (composite or SPS points), as labelled.

Residual-inflation proxies. We simulate a live-like error level by inflating the local residual. Proxy A uses $Y' = P + k(Y - P)$, with the real input windows. Proxy B uses $P' = Y + k(P - Y)$: it degrades only the FNO and scores the input-window estimators against the real target. k was fitted two ways. With k_rel = 1.665 the unmodified entry reproduces the live rel-L2 error. With k_T = 1.447 it reproduces the live band factor. Before v17 was submitted, all four proxies were computed (Table 3), and none was designated in advance. The decision record also noted that the proxies had overstated earlier band effects by 1.5–1.9×. After the live result, the AT proxy was closest for SPS (+4.01 vs +3.95 live). It over-predicted the composite by 18% and tke by 30%. That is one post-hoc agreement, not a validation. The ±5% default-band check (T 0.204 vs 0.208 live) reproduces a number that was known before k was fitted.

proxy (k)	Δrel	Δtke	Δmvpe	ΔSPS	Δcomposite
A (k_rel 1.665)	+0.08	+1.55	+0.07	+4.31	+1.40
B (k_rel)	+0.07	+0.57	+0.15	+6.98	+2.08
AT (k_T 1.447)	+0.07	+1.75	+0.06	+4.01	+1.33
BT (k_T)	+0.04	+0.94	+0.12	+5.59	+1.70
live (measured)	**−0.06**	**+1.34**	**+0.25**	**+3.95**	**+1.12**

Table 3. v17 minus v8: the four pre-submission proxy predictions and the live result.

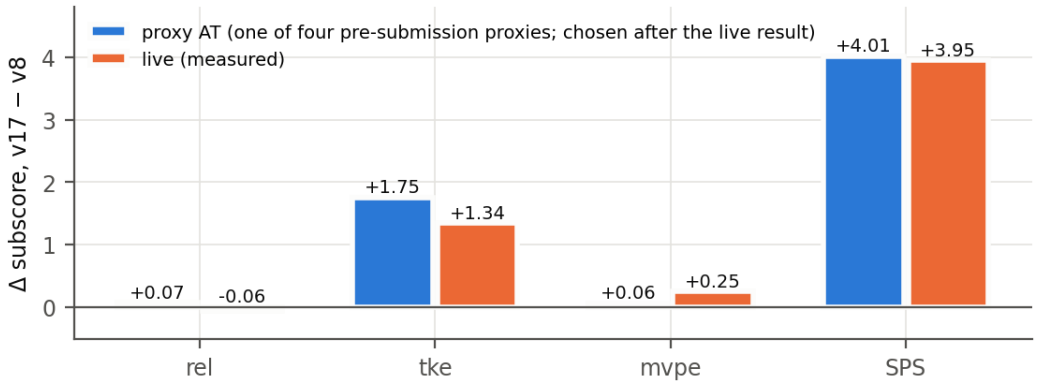


Figure 9. The AT row of Table 3 against the live result. AT was selected after the fact; tke is over-predicted by 30%.

6 Method: the shipped entry (v17)

Algorithm 1 specifies the method completely. Every constant is either given there or is a table entry released with the code (submissions/v17_per_window/_blend_tables.py). The prose below explains each step.

Algorithm 1 — v17 inference for one window (all statistics use this window only; per-window version v17pw)
Input $x \in R^{20\times32\times64\times3}$ (frames t = 1..20, channels u, v, p); NaN/±inf → 0.
Tables PRED, CENT $\in R^{20\times2\times6\times2}$ (lead × channel × sextile × (α, β)); WID $\in R^{4\times2\times10}$ (lead group × channel × decile).
1. $P \leftarrow FNO((x - \mu_{in})/\sigma_{in})\cdot\sigma_{out} + \mu_{out}$; P[..., p] ← 0. (official sim_real_ft checkpoint, fp32)
2. For each pixel and c ∈ {u, v}: $m = \#\{t : x_t \neq 0\}$; valid = (m > 0).
$E_m = \Sigma_t\, x_t$ / max(m, 1) (masked window mean); $E_l = x_{20}$ if $x_{20} \neq 0$ else E_m (last measured frame).
$s = \sqrt{(\text{mean}_t(x_t - \text{mean}_t\, x)^2)}$ (σ_{local}, unmasked); $d = \sqrt{(\text{mean}_{t=1..19}(x_{t+1} - x_t)^2)}$.
3. Ranks (per window, per channel, over valid pixels only): q_j = linear-interpolated quantiles at j/K;
bin = #{j : q_j < value}, i.e. searchsorted(q, value, left). K = 6 for s (sextile k), K = 10 for d (decile r).
4. For lead τ = 1..20 and valid pixels: (α, β) = PRED[τ, c, k]; $\hat{y}_\tau = E_m + \alpha(P_\tau - E_m) + \beta(E_l - E_m)$.
Invalid pixels (never measured in the window): $\hat{y}_\tau = P_\tau$. Constraint of the fit: α, β ≥ 0, α + β ≤ 1.
5. Interval centre: (α′, β′) = CENT[τ, c, k]; $\hat{c}_\tau = E_m + \alpha'(P_\tau - E_m) + \beta'(E_l - E_m)$ (same rule for invalid pixels).
6. Half-width: h_τ = WID[⌊(τ−1)/5⌋, c, r] (absolute, 0.0025–0.0256); lower = ĉ − h, upper = ĉ + h; p-channel bounds = 0.
7. Energy correction of the prediction only: $f = \hat{y} - \text{mean}_\tau\, \hat{y}$; $E_{pred} = \Sigma_{pixels,c}\, \text{mean}_\tau\, f^2$;
$E_{in} = \Sigma_{pixels,c}\, s^2$; $a = \text{clip}((E_{in}/E_{pred})^{1/4}, 1, 1.6)$; $\hat{y} \leftarrow \text{mean}_\tau\, \hat{y} + a\cdot f$. Bounds are not changed.
Output prediction ŷ (p = 0), lower, upper; float32. No interpolation between bins; ranks always fall inside the table, but widths are absolute and not rescaled for other flow conditions.

Fitting (offline, CPU, minutes). PRED: constrained least squares per cell on SELECT (4 trajectories, 168 windows) under proxy A ($Y' = P + k(Y - P)$, k = 1.534 fitted on SELECT to the live rel-L2 error). CENT: per-cell grid search maximising the SPS band factor T under proxies A and B. WID: per-cell h maximising $\exp(-2h/\sigma)\cdot\hat{F}(h)$ of the centre residual under proxies AT and BT (k = 1.394 on SELECT, fitted to the live band factor). The live v17 build pooled the rank edges over the 5 windows of each call; v17pw computes them per window (§13).

Base model. The official *sim_real_ft* FNO (50.36 M parameters, 99.9% in spectral convolutions; modes 4×12×16, width 64), stored in fp16 and run in fp32. No weights were changed.

Estimator blend. With E_m the masked input-window mean and E_l the last measured frame, each element is

$$\text{pred} = E_m + \alpha\cdot(P - E_m) + \beta\cdot(E_l - E_m),\ \alpha,\ \beta \geq 0,\ \alpha + \beta \leq 1$$

(α, β) are looked up per forecast lead (20) × channel (2) × σ_{local} sextile (6). They were fitted by constrained least squares on the SELECT split under proxy A. Figure 10 shows the fit. At lead 1, most of the field is persistence (α = 0 in sextiles 1–4). In the calmest sextile at later leads, the window mean takes over. In active regions the FNO keeps most of the weight.

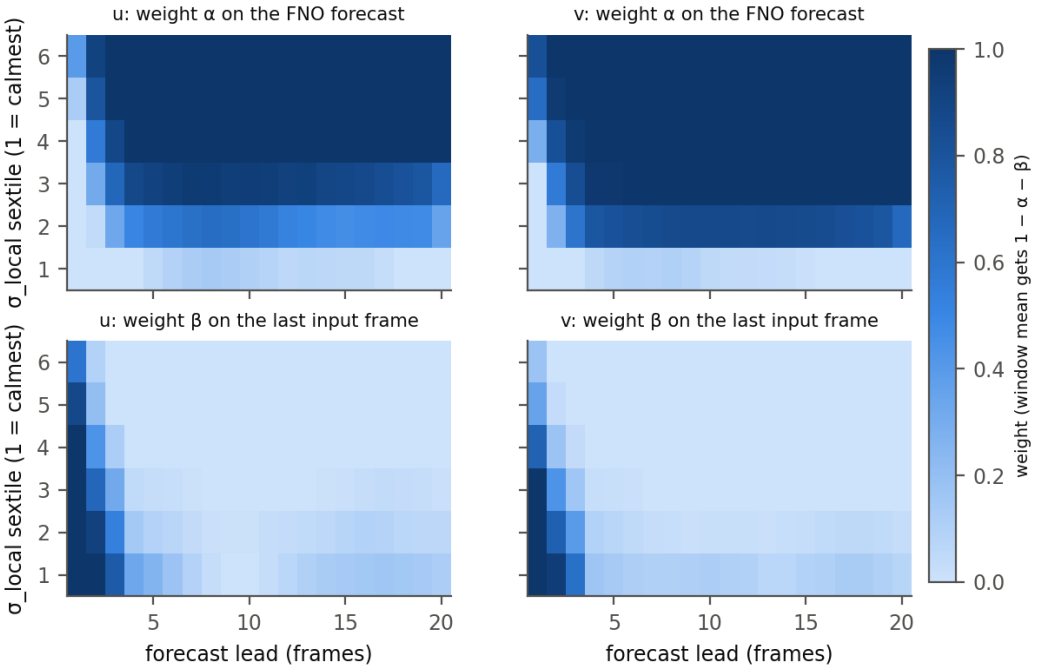


Figure 10. Fitted blend weights α (FNO) and β (last frame) by lead and σ_{local} sextile; the window mean receives $1 - \alpha - \beta$.

Why persistence wins at lead 1. On FINAL, the FNO's first frame has rel-L2 0.0583 against persistence's 0.0522. It is further from the second target frame (0.0749), so it is not a one-step time shift. Two defects combine. The FNO loses the last frame's fine-scale structure: its fine-scale content is weaker than the truth's (u 0.0124 vs 0.0145), and its fine-scale correlation with the truth is lower than persistence's (0.79 vs 0.92). It also jumps too far from the last frame: its lead-1 increment std is 1.24× the true one (1.50× in the high-pass band: 0.0089 vs 0.0059; increment correlation 0.54). In the calmest sextile its lead-1 mean squared error is 14× persistence's. By lead 2 the FNO wins (0.0598 vs 0.0913).

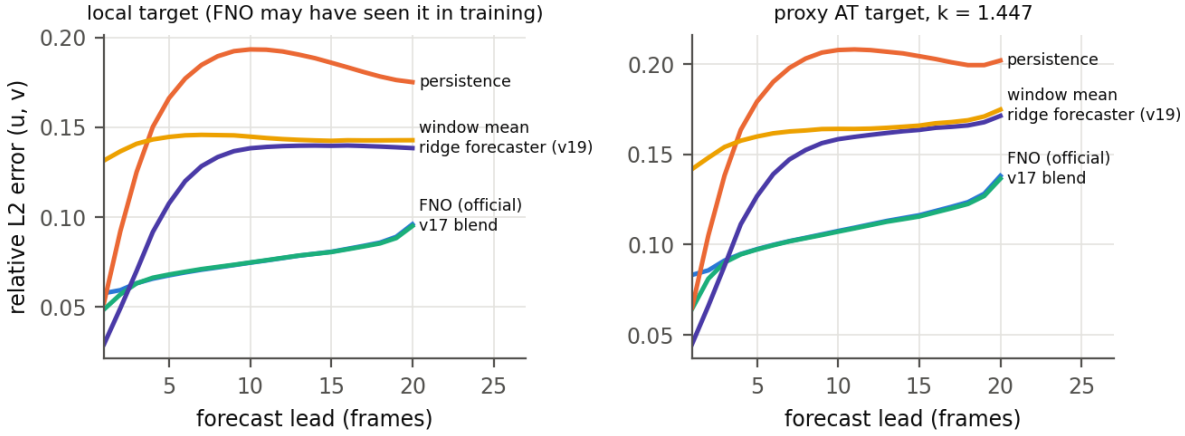


Figure 11. rel-L2 by forecast lead on FINAL for the FNO, v17, the ridge forecaster of v19 (§8), persistence and the window mean. Left: the real local target, which the FNO may have seen in training (Table 2). Right: the AT proxy target (k = 1.447).

Intervals. A second table of the same form sets the interval *centre*. Half-widths come from a table indexed by lead group × channel × decile of the input's one-step frame-difference RMS. It is one of the strongest single target-free features we found (Spearman 0.55 with |error|; others reached 0.62, and a learned combination 0.65). Each cell's width maximises $\exp(-2h/\sigma)\cdot\hat{F}(h)$. **These widths are absolute values (0.0025–0.0256).** The rank index is scale-free, but the widths are not. The intervals are tuned to one flow condition and carry no coverage guarantee. They are an SPS-optimised heuristic, not uncertainty quantification in the conformal sense.

Energy correction. The FNO forecast keeps about 60% of the target's temporal variance: 0.83 at lead 1, 0.50 at lead 20, with spatial correlation ≈ 0.9. For an MSE-optimal forecast this under-representation is expected, not a defect. We therefore describe the correction as fitting the estimator to the tke metric. Each window's deviation from its own 20-frame mean is scaled by $a = \mathrm{clip}((E_{in}/E_{pred})^{1/4}, 1, 1.6)$. The time mean, and so mvpe, is unchanged, and the intervals stay where they are.

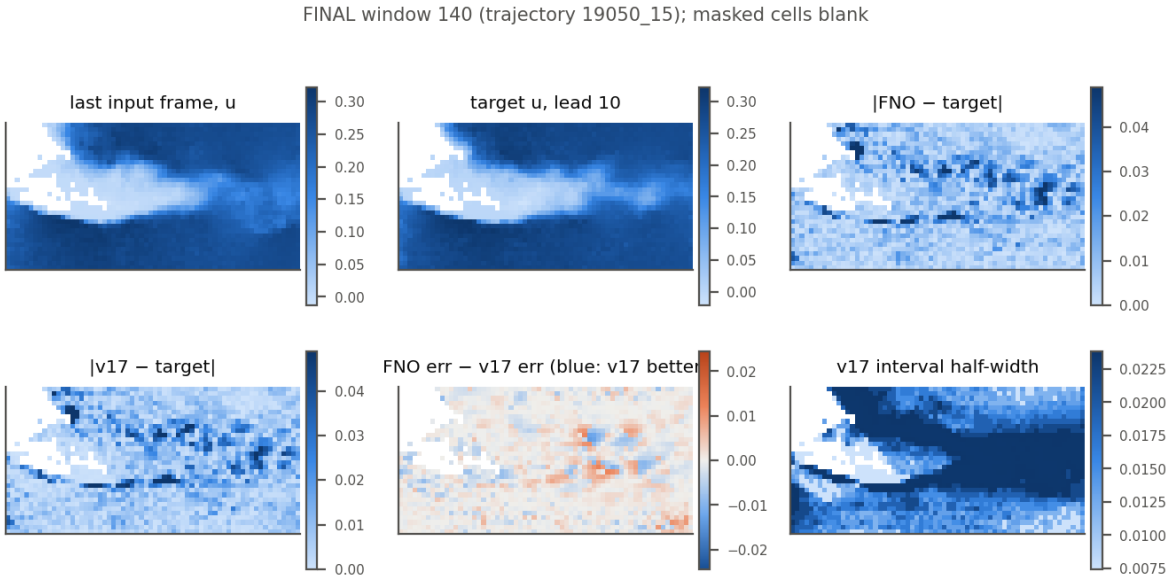


Figure 12. One median-difficulty FINAL window at lead 10 (u): input, target, FNO and v17 errors, their difference, and the v17 half-width map. Masked cells are blank.

7 Results

entry	rel	tke	mvpe	time	SPS	final	note
v8: official FNO + heteroscedastic bands	94.168	74.026	92.836	91.409	33.338	78.141	pooled bins
v17: v8 + blend, interval tables, energy correction	94.104	75.368	93.085	91.130	37.287	**79.261**	pooled bins
Δ	−0.064	+1.342	+0.249	−0.278	+3.949	+1.120	
exp1_sf: clean sim→real FNO + interval tables	93.728	77.348	92.378	91.701	37.160	79.238	per-window
1st place (benchmark)	94.746	80.191	94.243	92.030	44.967	82.151	

Table 4. Live development-phase results against the 1st-place benchmark. The v17 build pooled rank edges over the windows of each call (§13); its per-window rebuild was not scored live. The best live score without cross-window pooling is exp1_sf, 79.238 (§10); its tables were fitted on SELECT, so Disclosure 2 applies.

component, vs v8 (proxy AT)	Δrel	Δtke	Δmvpe	ΔSPS	Δcomposite
energy correction only (v16)	−0.04	+1.61	0	+0.30	+0.21
uniform ×1.2 widths only	0	0	0	+1.01	+0.29
interval-centre table only	0	0	0	+2.53	+0.72
frame-difference width table only	0	0	0	+1.78	+0.51
prediction blend + energy correction	+0.07	+1.75	+0.06	+0.43	+0.31
blend + centre + energy correction	+0.07	+1.75	+0.06	+2.99	+1.04
+ frame-difference width table = v17	+0.07	+1.75	+0.06	+4.01	+1.33

Table 5. Component ablation under proxy AT (experiments/estimator_blend/controls2_out.txt). Attribution depends on order: the width table adds +1.78 SPS alone but +1.02 when added last. The centre table is the largest single component either way.

Robustness. With per-window rank bins, v17's gain over v8 under proxy AT is positive on all 10 FINAL trajectories: Δcomposite mean +1.35 (SD 0.15, range +1.10 to +1.56), ΔSPS +3.2 to +4.6. We refitted the tables on four trajectories drawn at random (one each from 4 of the 5 AoAs, chosen without the live fingerprint). With the proxy scale k refitted on that split, the gain drops to +0.71 (10/10 positive). With SELECT's k it drops to +0.26 (8/10 positive on composite, 6/10 on SPS). On this split, 48–81% of v17's local gain on FINAL depends on fitting on flows in FINAL's own regime. FINAL was itself chosen as the trajectories nearest the live fingerprint, and the random split is lower in Reynolds number, so this bounds the value of regime match to FINAL, not the value of the logged information as such. A swap ablation (Table 6) locates it. Replacing only the width table costs 0.35–0.76, only the centre table 0.16–0.17, and only the prediction table 0.12–0.14. The random split's flows are lower-Reynolds and its widths 20–51% narrower (experiments/robustness/eval_randk_out.txt). The absolute widths are the largest single risk for the private test at unseen AoA and Re, but the centre and prediction tables are also condition-specific. FINAL was also used to compare many recipes and proxy variants, so it is a selection set, not a clean hold-out, and the

per-trajectory SD does not account for that selection.

tables used (FINAL, proxy AT)	Δ vs v8, random split, k refit	Δ vs v8, random split, SELECT's k
all fitted on SELECT (fingerprint)	+1.356	+1.356
swap prediction table only	+1.241	+1.221
swap centre table only	+1.195	+1.188
swap width table only	+1.002	+0.593
swap centre + width	+0.823	+0.385
all fitted on random split	+0.710	+0.255

Table 6. Which fitted component carries the dependence on the fitting split (experiments/robustness/swap_ablation_out.txt).

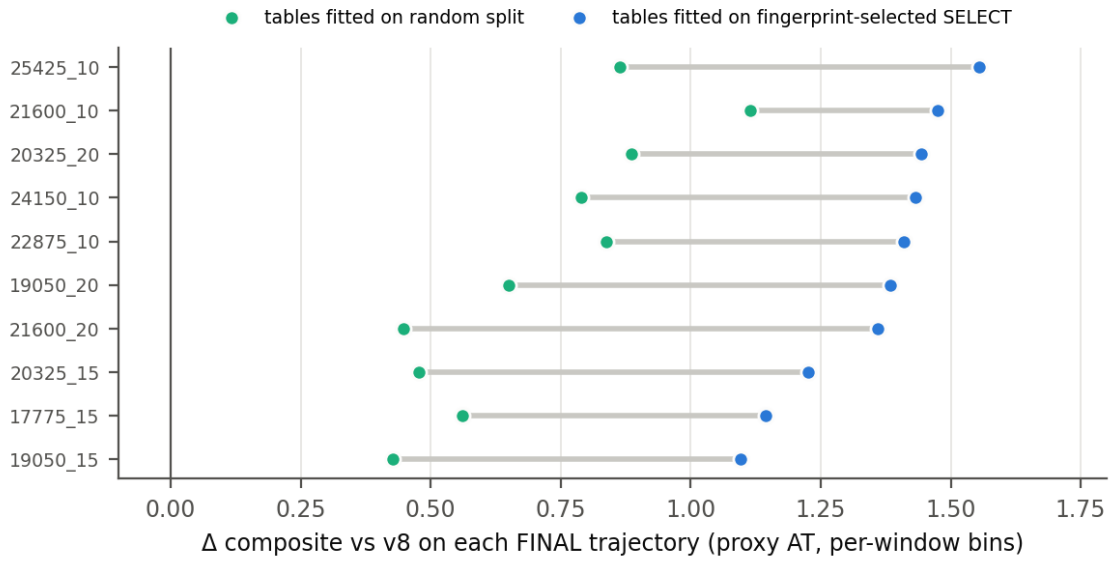


Figure 13. Per-trajectory Δcomposite over v8 (proxy AT) with tables fitted on the fingerprint-selected split vs a random AoA-stratified split.

Repeated over 13 random splits. A single random draw could make the share look larger or smaller than it is, so we repeated the refit on 13 AoA-stratified draws of 120–168 windows each (the first is the split above; experiments/robustness/k9_multi.py). With the proxy scale refitted on each split, the random-fit gain ranged +0.18 to +1.04 (mean +0.62); with SELECT's scale, -0.26 to +0.88 (mean +0.37). The share of v17's +1.36 that needs the regime-matched fit is therefore 23%–119% (median 62%; 52% with the scale refitted, 81% with SELECT's); above 100% means the random fit lost to v8. Every random fit with its own scale stayed positive (Fig. 14).

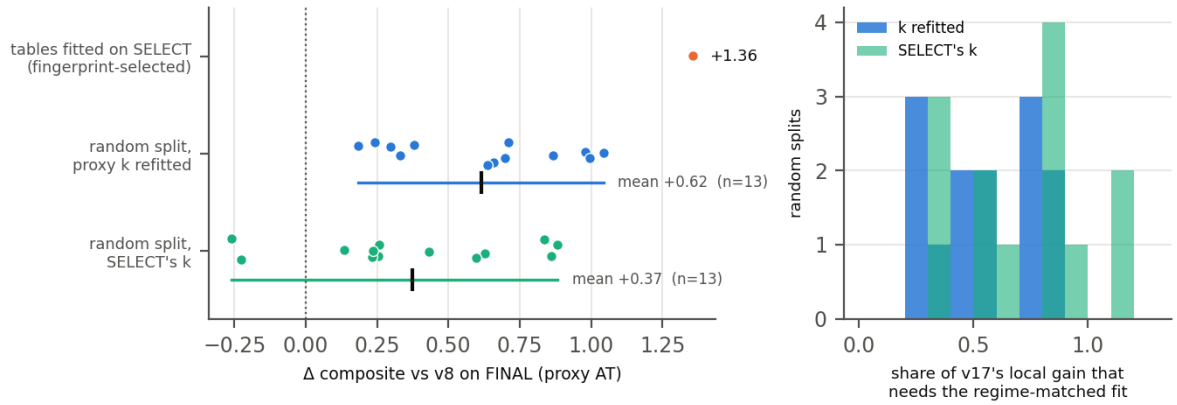


Figure 14. v17's recipe refitted on random AoA-stratified splits and scored on FINAL under proxy AT, against the fit on the fingerprint-selected SELECT split. Left: each dot is one split; bars show the range and mean. Right: the share of the SELECT-fit gain that the random fit does not recover.

Leave-one-AoA-out and leave-one-Re-band-out. The random splits still draw fit and test flows from the same mix of conditions. To test transfer to an unseen condition, we held out one angle of attack at a time (5 folds) and one Reynolds band at a time (3 folds: below 10k, 10–20k, above 20k), fitted v17's tables on 8 trajectories from the other folds (2 per angle, 3 independent draws per fold), and scored them on every

held-out trajectory against v8 under proxy AT (experiments/robustness/loo.py). For reference, the same tables were also fitted on the held-out fold itself (fewer windows per trajectory, so not an upper bound). Transferred tables gained +0.93 to +3.04 composite over v8 (fold means), against +1.39 to +2.94 when fitted in-fold, keeping 53–108% of the in-fold gain. Transfer beat v8 on every fold. The proxy scale is refitted on each held-out fold (k_T 1.44–2.01), so each gain is relative to that fold's own v8 baseline and gains are not comparable across folds; the largest gains come where v8's widths fit worst (v8 coverage 0.58–0.66). Interval coverage of the transferred tables ranged 0.69–0.88 (optimum 0.70–0.80), which is the calibration risk for unseen conditions (Fig. 15). This measures the tables, not the model: the official FNO very likely trained on every trajectory, so its held-out forecasts are better than a truly unseen condition would give, as in K9. A clean version retrains the forecaster per fold (8 folds × 3 seeds, about 24 Kaggle T4 GPU-hours); we have not run it.

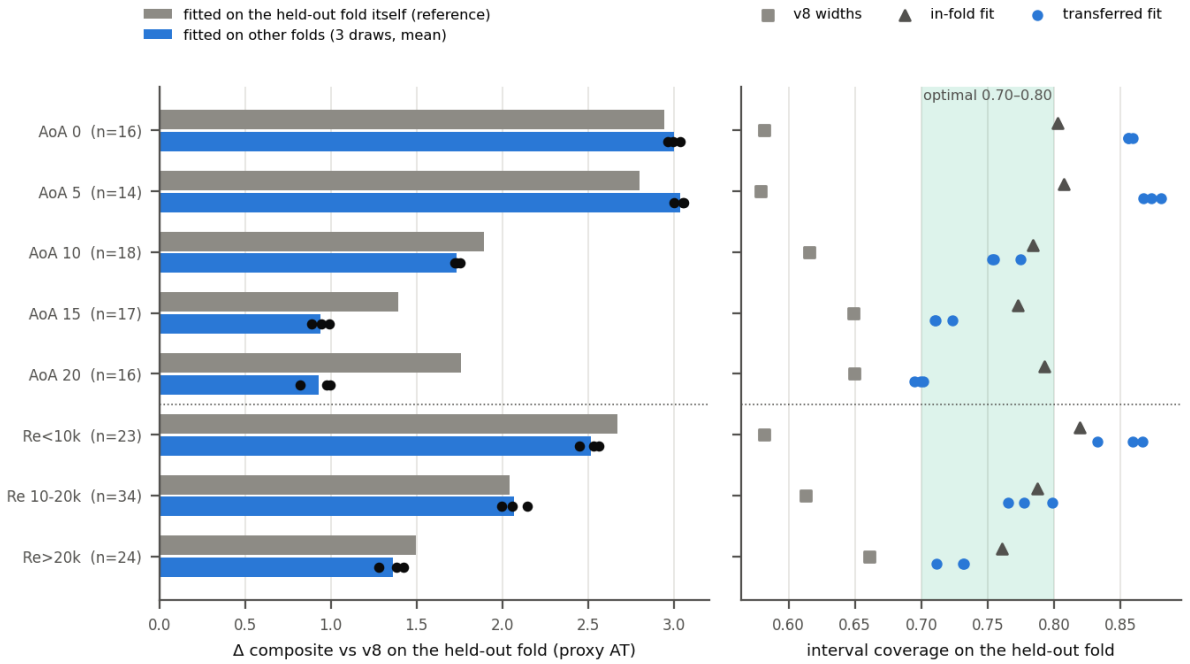


Figure 15. Leave-one-AoA-out (top) and leave-one-Re-band-out (bottom). Left: v17's tables fitted on other folds (blue, dots are the 3 draws) against tables fitted on the held-out fold itself (grey, reference), scored on the held-out fold under proxy AT. Right: interval coverage on the held-out fold for the v8 widths (squares), the in-fold fit (triangles) and the transferred fits (dots); the shaded band is the SPS-optimal 0.70–0.80.

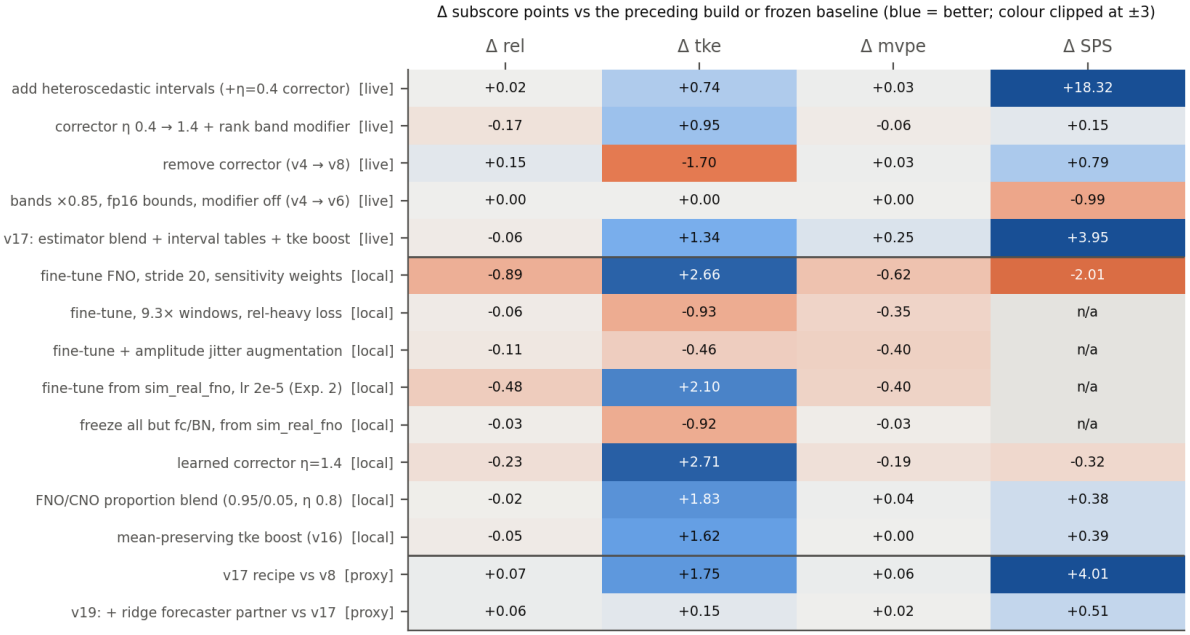


Figure 16. Effect of each method on each subscore against its predecessor or frozen baseline (live, local-memorised, or proxy, as tagged). Training rows repeatedly buy tke and sell rel. Only interval and blend changes raised SPS without costing accuracy.

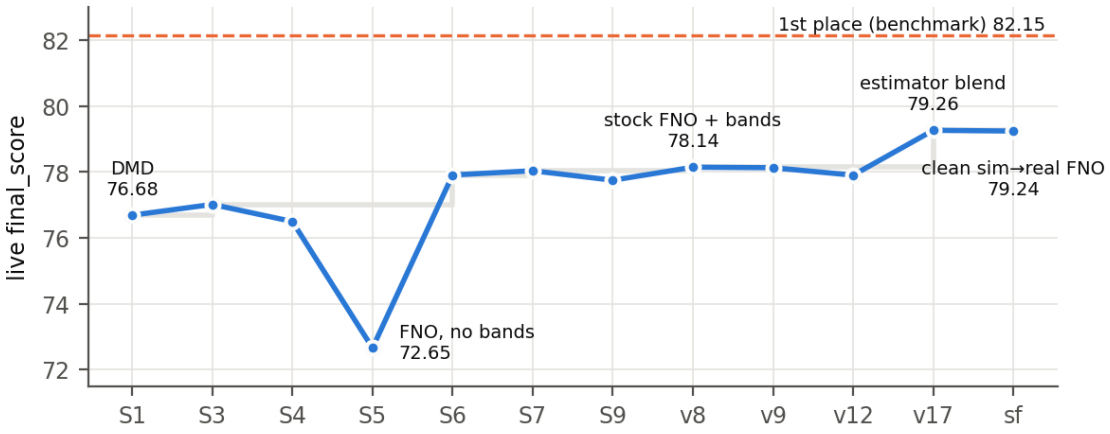


Figure 17. Live final_score of the submissions whose scores are recorded, in order. S2 is omitted (only its SPS was recorded). S8, a CUDA-graph build, crashed at import (score 0) and is not plotted. The grey step is the best so far. Final scores for v7, v10, v11, v13 and v15 were not recorded (v10's time subscore, 83.09, is in Appendix A). The dashed line is the 1st-place benchmark. sf = exp1_sf (clean sim→real FNO).

8 How we decided: rules and the protocols of the key experiments

condition observed	decision	where it was applied
improves held-out rel, tke and mvpe together	keep, and move towards a live test	FT10 (from sim-only)
improves tke but lowers rel or mvpe (amplitude trade)	reject	FT01, FT09, corrector η 1.4 (K2, K3)
improves only on a split the reference model may have memorised	do not trust; re-test on an honest split or a proxy	log-POD, rank modifier, v18 (−0.055 real, +0.13 proxy)
positive under all four proxies (A, B, AT, BT)	candidate for a live slot	v17, v19
positive under some proxies only	do not submit	v18 unconstrained blend (BT, B negative)
gain smaller than run-to-run noise (±0.017 final, runtime only)	do not submit	runtime micro-optimisations
runtime cost exceeds composite gain (0.093 per time point)	reject	temporal-disagreement band feature (−1.47 time)
SPS gain without coverage collapse (T up; coverage near the 0.70–0.80 optimum, not far below)	candidate	v17 (coverage 0.84 on the memorised pair with the real target; 0.80 on FINAL and 0.76 on SELECT under proxy AT)
live gain confirmed	adopt as incumbent	v8, then v17
any build	must pass preflight; per-window builds also the independence test	all submissions

Table 7. Decision rules as applied. The final slot choices were also routed through a logged decision gate (tools/decisions/).

The cards below give the full protocol of the twelve experiments that shaped the campaign. Every other experiment is in Appendix B, with the same fields in compact form.

K1 Heteroscedastic prediction intervals (S5 → S6, live)	
Purpose	Is the default ±5% interval, used when no bounds are returned, what is holding SPS down?
Hypothesis	Intervals proportional to each pixel's input variability raise the band factor T, and so SPS, without touching accuracy.
Criterion	Live SPS rises. Not pre-registered.
Intervention	Return lower/upper = $\hat{y} \pm (A + K \cdot \sigma_{local})$, $A_u = 0.004$, $A_v = 0.003$, K = 1.5 (scale bracketed earlier on three live DMD builds). Bundled with a learned corrector at η = 0.4.
Baseline	S5: official FNO, no bounds (default ±5%).
Data / evidence	Live board (AoA 15 evaluation set).
Result	SPS 14.08 → 32.40 (+18.3); T 0.208 → 0.475; final 72.65 → 77.90. rel +0.02, tke +0.74 (the corrector's share).
Conclusion	Interval construction dominates SPS. This was the largest single gain of the campaign.
Caveat	Two changes at once. The corrector's accuracy effect is small next to the interval effect.

K2 Learned residual corrector (v4 vs v8, live)	
Purpose	Does a small learned operator trained on the FNO's residuals add composite score?
Hypothesis	Its tke gain outweighs any rel/SPS loss.
Criterion	Live final_score of the corrected build exceeds the uncorrected one.
Intervention	$\hat{y} = P + \eta \cdot C(x)$, C a 668,316-parameter TinyOperator (zero-initialised output) trained on real residuals; $\eta = 1.4$ (v4) vs $\eta = 0$ (v8). Identical intervals.
Baseline	v8 ($\eta = 0$), otherwise identical.
Data / evidence	Live board; local FINAL for the pre-test.
Result	v4 − v8 live: rel −0.148, tke +1.699, mvpe −0.029, SPS −0.789, final −0.108. Local-to-live transfer of the three accuracy deltas: 66% / 63% / 15%.
Conclusion	The corrector buys tke with rel. Because the intervals were centred on the corrected field, SPS fell. Closed.
Caveat	Only two η values were shipped live.

K3 Further fine-tuning of the official FNO (FT01, FT02)	
Purpose	Can more real-data training improve the official checkpoint?
Hypothesis	Fine-tuning on *train_real* (hold-out excluded) with a composite-aligned loss improves held-out rel, tke and mvpe.
Criterion	All three ≥ 0 on FINAL.
Intervention	FT01: stride 20 (3,362 windows), sensitivity-weighted loss. FT02 run B: stride 2 (31,343 windows), loss weights rel 0.8 / tke 0 / mvpe 0.2, input noise $\sigma = 0.02$, 7575_0 excluded, Kaggle T4. FT02 + amplitude jitter 0.70.
Baseline	Frozen official FNO on the same FINAL windows.
Data / evidence	Local; FINAL may have been seen by the baseline (Table 2).
Result	FT01 Δ −0.89 / +2.66 / −0.62 (SPS −2.01). FT02-B Δ −0.06 / −0.93 / −0.35. With jitter Δ −0.11 / −0.46 / −0.40. About 100% of the movement happens in epoch 0.
Conclusion	From this converged start, the only direction that lowers the training loss is an output-amplitude rescale. It buys tke and sells rel. Forbid it and training overfits.
Caveat	Judged against a possibly memorised baseline. Never submitted live, so not a clean negative.

K4 Clean-start sim-to-real fine-tune (Exp. 1)	
Purpose	How much real-data transfer does our own training achieve on trajectories no model has seen?
Hypothesis	Fine-tuning the simulation-only FNO on 67 real trajectories lifts held-out scores far above the simulation-only start.
Criterion	Beats the frozen sim-only FNO on all three subscores on FINAL.
Intervention	Start from sim_fno.pth; all parameters trainable; lr 1e-4; 80 epochs; composite-derivative loss weights; pool train_pool_v12_3 (67 trajectories, hold-out and 7575_0 excluded); Kaggle T4.
Baseline	Frozen sim_fno on the same 10 FINAL trajectories.
Data / evidence	Local, honest (FINAL never seen by either model).
Result	rel/tke/mvpe 79.71 / 66.87 / 75.20 → 94.27 / 78.78 / 95.51. Plateau from epoch 10.
Conclusion	A large genuine sim-to-real gain.
Caveat	Local FINAL is not the live set. Never submitted live.

K5 Output-layer-only fine-tuning (FT10, FT11)	
Purpose	Does freezing the spectral weights remove the amplitude trade of K3?
Hypothesis	With only the lift/projection and BatchNorm layers trainable, rel cannot be sold for tke.
Criterion	All three subscores ≥ the frozen start on FINAL.
Intervention	Train only fc0, fc1, fc2 and BatchNorm affine (9,667 of 50.36 M parameters) with radial spectral input augmentation; 30 epochs; epoch chosen on SELECT. FT10 starts from sim_fno; FT11 from the official FNO (lr 2e-5).
Baseline	The respective frozen start.
Data / evidence	Local; honest for FT10, possibly memorised for FT11.
Result	FT10: +12.75 / +5.07 / +18.33 (all three up). FT11: −0.03 / −0.92 / −0.03.
Conclusion	Freezing works when the start has headroom. It only degrades an already-calibrated output layer.
Caveat	The two effects of augmentation and freezing were not separated.

K6 Is our hold-out unseen by the official model?	
Purpose	Decide whether local comparisons against the official FNO can be trusted.
Hypothesis	If the hold-out were unseen, the official model would score no better there than on the live set.
Criterion	Official hold-out score > own live score ⇒ the hold-out is not independent of its training.
Intervention	Score frozen sim_fno, sim_cno and the official sim_real_fno on the same 10 FINAL trajectories (420 windows, stride 20).
Baseline	The official model's own live score.
Data / evidence	Local + live.
Result	Official 96.38 / 81.04 / 97.06 locally vs 94.17 / 74.03 / 92.84 live. sim_fno 79.71 / 66.87 / 75.20; sim_cno 78.15 / 67.15 / 74.74. From FINAL to live the official raw FNO drops −2.21/−7.01/−4.23, the clean Run 1 only −0.43/−1.83/−3.13.
Conclusion	Consistent with memorisation (Table 2). We stopped using the official model as a local yardstick.
Caveat	A harder live regime would also produce a gap.

K7 Input-window estimator blend and interval tables (v17, live)	
Purpose	Can input-window statistics identify where trivial estimators beat the FNO, and place intervals better?
Hypothesis	Blending towards persistence and the window mean in calm regions and short leads lowers error where SPS is most sensitive; re-centred, variability-indexed intervals raise T.
Criterion	Positive composite under all four proxies before submission (Table 3); then live.
Intervention	Algorithm 1 on top of v8, with the tables fitted on SELECT.
Baseline	v8 (live 78.141).
Data / evidence	Proxy (FINAL), then live.
Result	Proxies +1.33 to +2.08 composite. Live: final +1.120 (78.141 → 79.261), SPS +3.95, tke +1.34, mvpe +0.25, rel −0.06, time −0.28. The board is deterministic for accuracy and SPS; sampling uncertainty over the ~90 evaluation windows of one flow condition is not quantified.
Conclusion	Adopted. The mechanism is decomposed in K8.
Caveat	The live build pooled rank edges over five windows (§13). The tables were fitted on a fingerprint-selected split (K9).

K8 Component ablation of v17	
Purpose	Which parts of v17 cause the gain?
Hypothesis	Most of the SPS gain comes from interval placement, not from the prediction.
Criterion	Not pre-registered.
Intervention	Add components one at a time to v8: energy correction; uniform width ×1.2; centre table; prediction blend; widths table (Table 5).
Baseline	v8.
Data / evidence	Proxy AT on FINAL.
Result	Centre table alone +2.53 SPS (+0.72 composite); widths alone ×1.2 +1.01 SPS; prediction blend + energy correction +0.43 SPS, +1.75 tke; full v17 +4.01 SPS, +1.33 composite.
Conclusion	Moving the interval centre is the largest single cause.
Caveat	Persistence-only and mean-only blends were not run separately.

K9 Fingerprint-free refit	
Purpose	How much of v17's gain depends on fitting the tables on flows chosen to resemble the evaluation set?
Hypothesis	The prediction part transfers; the absolute widths do not.
Criterion	Not pre-registered.
Intervention	Refit all tables (per-window bins) on four random trajectories, one each from 4 of the 5 AoAs (seed 0: 10125_5, 12675_10, 7575_15, 3750_20; 168 windows); refit proxy k the same way (1.413 / 1.826).
Baseline	The same recipe fitted on SELECT.
Data / evidence	Proxy AT on FINAL, per trajectory.
Result	k refitted: +0.71 vs v8 (10/10 positive), −0.65 vs SELECT-fitted. SELECT's k: +0.26 (8/10 composite, 6/10 SPS), −1.10. Swap ablation (Table 6): width table 0.35–0.76, centre 0.16–0.17, prediction 0.12–0.14. Random-split widths 20–51% narrower (lower Re).
Conclusion	48–81% of v17's local gain on FINAL (first split; 23%–119% over 13 splits, median 62%) depends on fitting on flows in FINAL's regime (FINAL was itself fingerprint-selected, so this measures regime match, not the logged information as such); the widths carry most of it, the centre and prediction tables the rest.
Caveat	The first split is reported here; over 13 random splits the share is 23%–119% (median 62%; 52% with the scale refitted, 81% with SELECT's; §7).

K10 Why persistence beats the FNO at lead 1	
Purpose	Explain the α = 0 cells of the blend table (Fig. 10).
Hypothesis	Either the FNO output is shifted by a time step, or it over-smooths.
Criterion	A time shift ⇒ P_1 is closer to Y_2 than to Y_1.
Intervention	Compare P_1 with Y_1, Y_2 and persistence; compare fine-scale content and increment correlations; split by σ_{local} sextile.
Baseline	Persistence x_{20}.
Data / evidence	Local FINAL, real target.
Result	rel-L2 of P_1: 0.0583 vs Y_1, 0.0749 vs Y_2; persistence 0.0522. Fine-scale correlation 0.79 vs persistence 0.92. In the calmest sextile the FNO's error is 14× persistence's. Lead 2: FNO 0.0598 vs persistence 0.0913.
Conclusion	Lost fine-scale structure plus an overshooting first increment; not a one-step time shift.
Caveat	Measured on data the FNO may have memorised, which favours the FNO.

K11 Energy correction for the tke metric	
Purpose	Can the tke deficit be reduced without costing rel?
Hypothesis	The forecast's variance map is well placed but too weak, so a mean-preserving amplitude increase raises tke with little rel cost.
Criterion	Pre-registered primary: fixed gain a = 1.15 (experiments/tke_energy/PREREG.txt).
Intervention	Scale fluctuations about the forecast's own time mean by a = clip($(E_{in}/E_{pred})^{1/4}$, 1, 1.6). Intervals unchanged.
Baseline	Official FNO with v8 intervals.
Data / evidence	Local FINAL, then live inside v17.
Result	Diagnosis: predicted/target energy 0.62, spatial correlation 0.90. Fixed a = 1.15: tke +1.38, rel −0.07. Adaptive: tke +1.62, rel −0.05, SPS +0.39.
Conclusion	This fits the estimator to the metric: an MSE-optimal forecast under-represents variance by design.
Caveat	tke is a 20-frame variance with effective n ≈ 4; the gain is metric-specific.

K12 Learned linear forecaster as a blend partner (v19)	
Purpose	Can a simple model trained only on released data beat the window estimators, honestly?
Hypothesis	A local linear forecaster beats persistence and the window mean at short leads.
Criterion	Positive under all four proxies.
Intervention	Ridge regression per (σ_{local} sextile × 4×4 spatial block) from the last 10 frames in a 5×5 u,v neighbourhood plus the window mean (549 features), λ = 10^{-4}, shrunk towards the per-sextile fit; trained on the 67-trajectory pool at stride 4 (13,629 windows); added to the blend as a third, constrained partner.
Baseline	v17.
Data / evidence	Local honest (standalone), proxy (blend).
Result	Standalone FINAL rel-L2 0.126 vs window mean 0.141 and persistence 0.170. Best predictor at leads 1–2. Blend: +0.16 to +0.33 composite over v17 under the four proxies.
Conclusion	Built as v19pw (independence PASS); never submitted.
Caveat	Never evaluated live. Runtime on the evaluation GPU unmeasured.

9 Negative results, reversals, and the last candidates

Two conclusions we had to reverse. Early in the campaign we closed blending “as a class”: a 3,624-combination search over FNO, CNO, DMD, persistence and window-mean blends returned the pure FNO. We also concluded that the intervals sat at their analytic optimum, worth at most +0.15 composite. v17 contradicts both, with SPS +3.95 live from blending and interval placement. Both early analyses were run on the memorised split, where the FNO's errors look about 1.5× smaller than they are live. That makes every trivial estimator look worse and every tight interval look adequate. The lesson is general: a conclusion drawn on a biased yardstick inherits the bias.

direction	evidence	reading
learned corrector on FNO output (668 k params)	live 78.03 (η 1.4), 77.90 (η 0.4) vs 78.14 at η 0	gains tke, moves interval centre, loses SPS
test-time BatchNorm adaptation	−0.81 local, 5/5 negative	no covariate shift to correct
log-POD variance denoising	local +2.73 → live +0.03	memorised-split artefact
narrower intervals ×0.85 with modifier off (v6)	live −0.99 SPS	confounded: two changes at once
torch.compile / AOT / TF32 / CUDA graphs	PermissionError; 3–6×10^{-3} drift; import crash (0)	container limits; spectral weights brittle
FFT padding retune	−3.97 to −22.6 (sum of three)	model welded to its padding
richer interval-width models (ridge, GBM)	≤ +0.02 composite over the lookup	band shape saturated
DMD / linear trend as extra blend partners	+0.00 to +0.02	no information beyond window mean

Table 8. Further closed directions (full record in Appendix B).

Candidates never evaluated live. v19 adds a ridge-regression forecaster (last 10 frames × 5×5 neighbourhood, trained on 67 trajectories) as a third blend partner. It is honest by construction and beats every other predictor at leads 1–2 (Fig. 11). The proxies put it at +0.16 to +0.33 over v17. v18ft applies the v17 post-processing to our fine-tune of the official model.

10 A clean sim-to-real model, measured in the live regime

Everything above post-processes the official FNO, and its local scores cannot be trusted (§5). To have a model we can evaluate honestly, we retrained Exp. 1 (Run 1: sim_fno → 67 real trajectories, lr 1e-4, 12 epochs, best epoch 8 by SELECT). We also trained one variant (Run 2) with a learned residual path to the last input frame. Both exclude SELECT and FINAL, so their local scores are honest.

model (never saw SELECT/FINAL)	SELECT sum	FINAL rel	FINAL tke	FINAL mvpe	FINAL sum
sim_fno (start)	220.12	79.71	66.87	75.20	221.78
Exp. 1	–	94.27	78.78	95.51	268.56
Run 1 (retrain)	266.81	94.17	79.29	95.54	269.00
Run 2: + residual-to-last-frame	266.82	94.20	79.29	95.55	269.04

Table 9. Clean sim-to-real FNO fine-tunes. The retrain reproduces Exp. 1. The residual path learned to lean on the last frame for v at short leads (gate 0.58 at lead 1) and hardly for u (0.07), but changed the scores by +0.04 of 269. Table 9 gives the Kaggle evaluation of the fp32 checkpoints; §10 and Appendix A use our local re-evaluation of the packed build (FINAL 94.16 / 79.18 / 95.50), which differs by at most 0.11. Seed variance was not measured; Exp. 1 and Run 1 differ by 0.45 in sum, so +0.04 is not detectable.

Post-processing a clean model. Refitted honestly (k = 1) on Run 1, the v17 prediction blend *hurts*. It pulls the forecast toward the window mean and cuts tke from 79.2 to 71.6. The tke boost repairs only part of that. The blend compensated for the official FNO's lead-1 over-smoothing, which the clean model does not share. The interval-centre and width tables still help (SPS 33.0 → 37.7). The build therefore uses the raw forecast plus the interval tables. A per-window scale for the widths, $h \propto \sqrt{(\text{median } \sigma_{\text{local}})}$, gave +0.13 composite on FINAL (+0.12 on its low-Re AoA-15 trajectories) but only +0.006 in-sample on SELECT, the live regime (Fig. 18). We shipped it as exp1_sf. Fully scale-free widths lost on SELECT both in-sample (coverage 0.747, −0.792 vs −0.760) and when fitted on FINAL, where they over-covered (0.832).

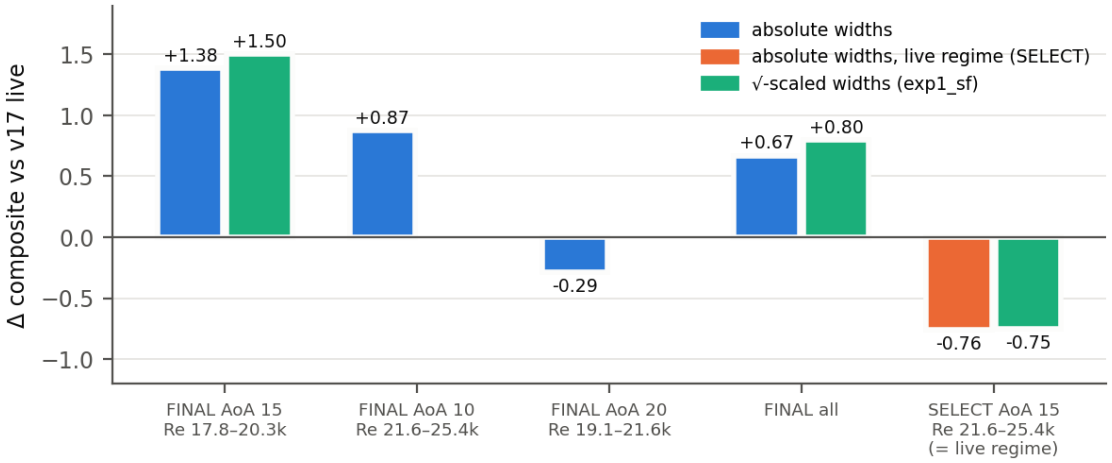


Figure 18. Run 1 build vs v17's live score by flow regime, on trajectories it never saw. It wins on FINAL's lower-Re AoA-15 trajectories (+1.38) and at AoA 10 (+0.87), but loses at AoA 20 (−0.29) and in the live regime (SELECT: AoA 15, Re 21.6–25.4k; −0.76), there mostly on rel (−0.9) and SPS (−2.8), while gaining tke (+3.1) and mvpe (+1.9). √-scaled widths were evaluated only where their bars are shown.

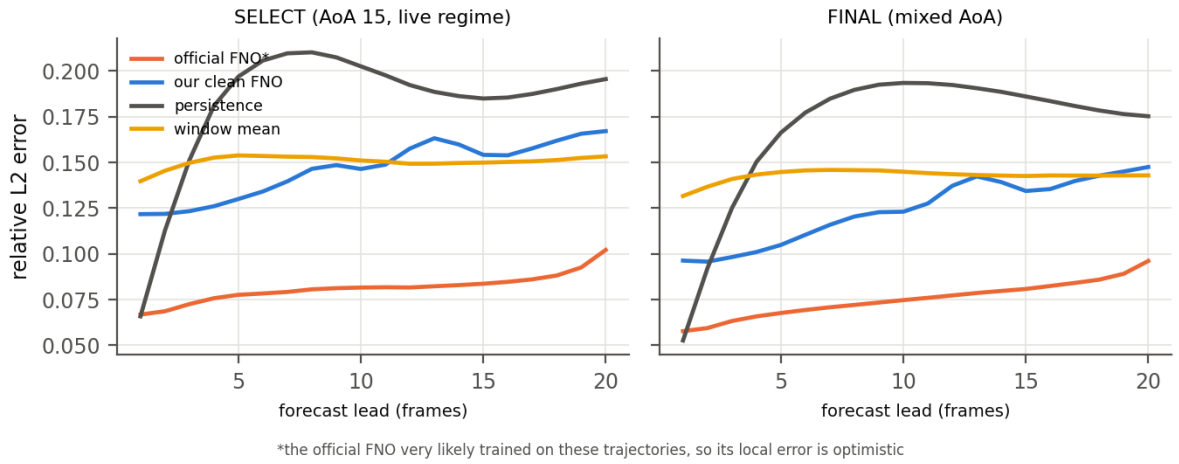


Figure 19. Error by forecast lead of our clean FNO against the official FNO, persistence and the window mean, on SELECT (live regime) and FINAL. Our model never saw these trajectories; the official one very likely did.

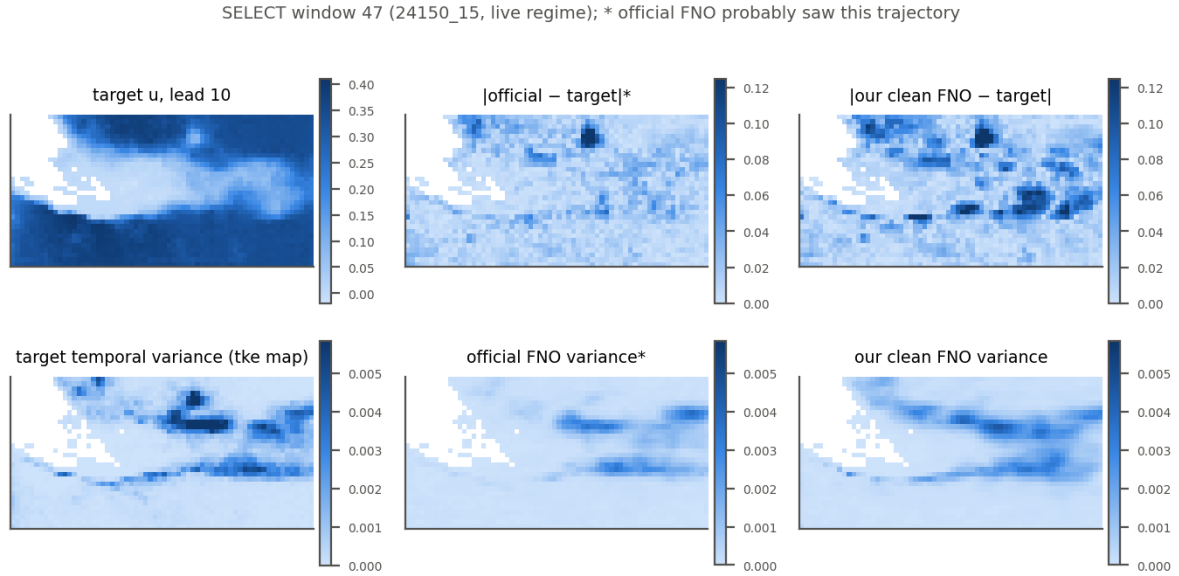


Figure 20. One live-regime SELECT window (u): target at lead 10, the two models' errors, and the temporal-variance (tke) maps. Our clean FNO's variance map is stronger but more diffuse, and its pointwise errors in the wake are larger.

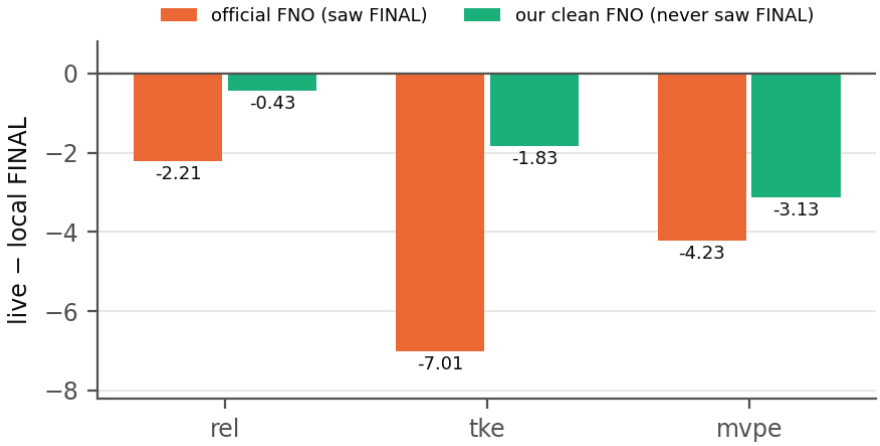


Figure 21. Score change from local FINAL to live. The official FNO, which very likely trained on FINAL, loses far more than our clean FNO, which never saw it.

Uncertainty and baseline arms. To put error bars on the local comparisons, we resampled whole trajectories (4,000 bootstrap draws; experiments/robustness/arms_bootstrap.py) and added two model-free arms, persistence and the window mean (Fig. 22). On FINAL (10 trajectories), our clean FNO minus the official FNO is -2.22 / -1.86 / -1.56 (rel / tke / mvpe; 95% intervals [-2.63, -1.75] / [-3.75, -0.25] / [-1.95,

-1.16]); the official model's local lead is expected if it trained on these trajectories. The memorisation test is the FINAL→live change. The clean model loses less than the official one by +1.78 / +5.18 / +1.10, with 95% intervals [+1.30, +2.19] / [+3.58, +7.08] / [+0.70, +1.50]; the share of bootstrap draws in which it loses less is 1.00 / 1.00 / 1.00. The interval covers only trajectory sampling on FINAL, not seed or live-sampling variance. On SELECT (4 trajectories, so the bootstrap is coarse) both networks are nominally ahead of persistence and the window mean on rel and mvpe; our clean FNO leads the window mean by only +0.14 rel and +0.09 mvpe, within overlapping intervals. Both model-free arms score 66.67 on tke by construction: a time-constant forecast has no temporal variance.

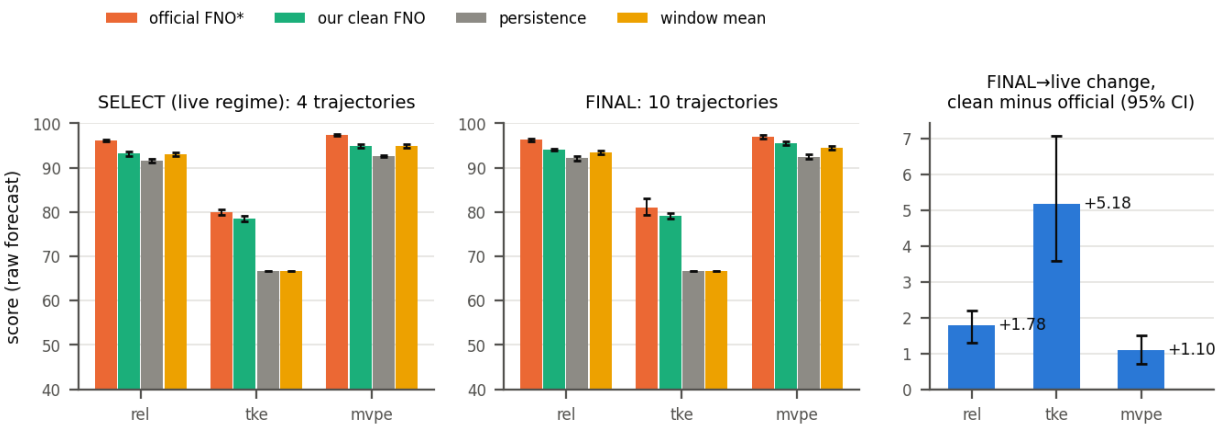


Figure 22. Raw-forecast scores of four arms on SELECT and FINAL with 95% trajectory-bootstrap intervals (*the official FNO very likely trained on these trajectories), and the FINAL→live change of our clean FNO minus that of the official FNO: positive means the clean model lost less between local and live.

Two more negative results. The official CNO, never scored live, trails the official FNO + v17pw by 3.1 composite on SELECT under the same proxy (2.7 on FINAL) and runs about 3× slower. An average of the official FNO and Run 1 never beats the official FNO alone on SELECT (+0.42 at 50/50 vs +0.67 alone, accuracy part), and two FNOs would exceed the 256 MB limit. On the live-like set we have no evidence that anything we built beats the official FNO + v17-style post-processing. This rests on proxy AT, which is unvalidated for cross-model comparisons (Limitation 6); live, the clean model reached parity (−0.023).

Live test. The √-width Run 1 build (exp1_sf) was submitted live: the first live score of a model that never saw the local data. The regime analysis predicted it below 79.26 (SELECT −0.75 for the √-width build; −0.76 with absolute widths, Fig. 18), while FINAL as a whole predicted above (+0.80). Result: rel 93.728 / tke 77.348 / mvpe 92.378 / time 91.701 / SPS 37.160, final 79.238.

What the live test shows. 79.238 is 0.023 below v17: better tke (+1.98) and runtime (+0.57), worse rel (−0.38) and mvpe (−0.71). SELECT predicted −0.75 against v17; live was −0.023. The miss is almost entirely SPS (SELECT 34.54, live 37.16), even though the tables were fitted on SELECT. The rel/tke/mvpe shifts from SELECT to live (+0.52/−1.17/−2.63) resemble the numpy DMD's (+0.55/−1.04/−1.86): a two-model consistency check, not a calibration. It is weaker than it looks: the local pool behind the DMD's SELECT numbers was not recorded. On the live set, the clean fine-tune reaches composite parity with v17 (the organisers' model plus our post-processing) with a different profile (more tke, less rel and mvpe). Its FINAL→live drop (−0.43/−1.83/−3.13) is far smaller than the official model's (−2.21/−7.01/−4.23), the strongest evidence in this report for the memorisation inference.

Not tested (future work). Clean fine-tunes of CNO (the run stalled at 5.8 s/batch and then ran out of memory on the T4) and of Transolver; DPOT-S/-L and UNet fine-tuned at 32×64 (only scored off the shelf, 88.8/70.1/84.4 on SELECT); WDNO and other architectures; using train_sim in our own training; an FNO trained on all 81 trajectories; multi-seed ensembles; SWA of the output-layer-only run; intermediate freeze ratios from the official FNO; Fisher-protected fine-tuning (PhysGuard); a learned interval-width head; directly optimised asymmetric intervals; compressing two models under 256 MB.

11 Figure atlas of our work

The remaining figures document the experiments of §4–§10 visually. Each caption says what was run and what it shows.

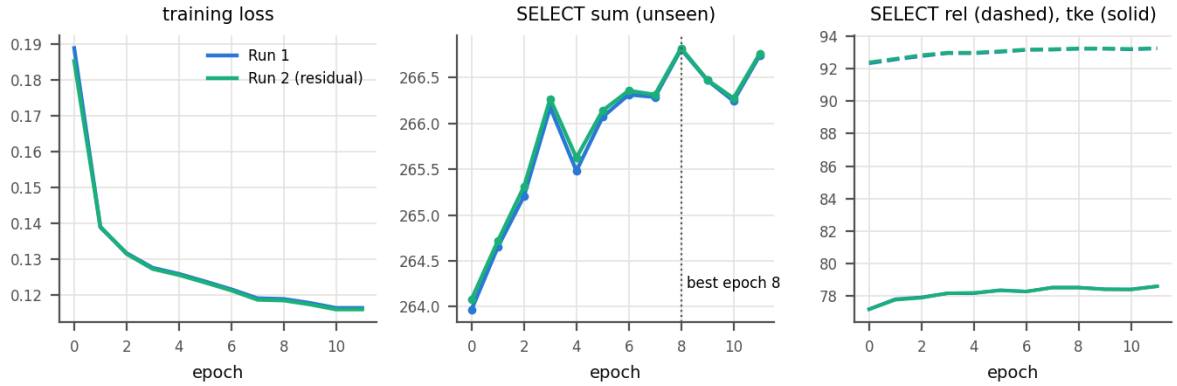


Figure 23. Kaggle training curves of our two clean sim-to-real runs: training loss, SELECT sum (unseen) and SELECT rel/tke by epoch. Nearly all the gain comes in epoch 0; both plateau from epoch 3 and peak at epoch 8.

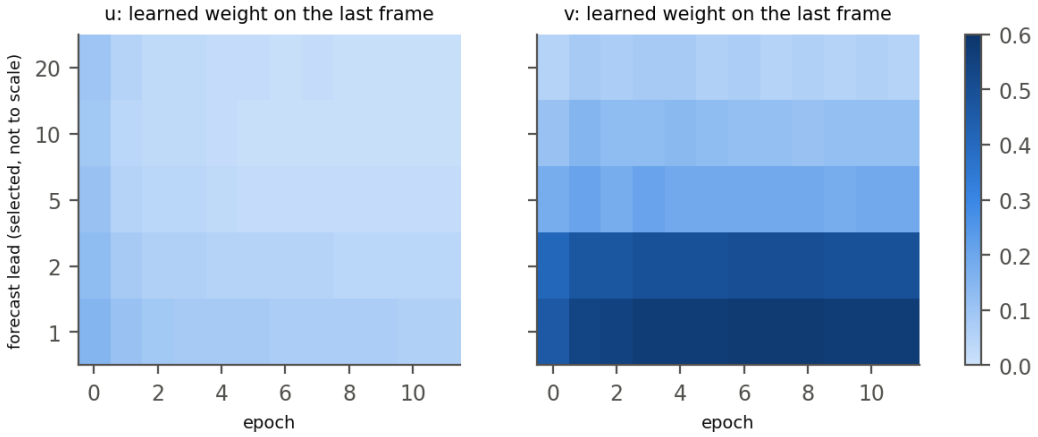


Figure 24. Run 2's learned weight on the last input frame by lead and epoch. It uses the last frame strongly for v at short leads and hardly for u. It converges by epoch 3.

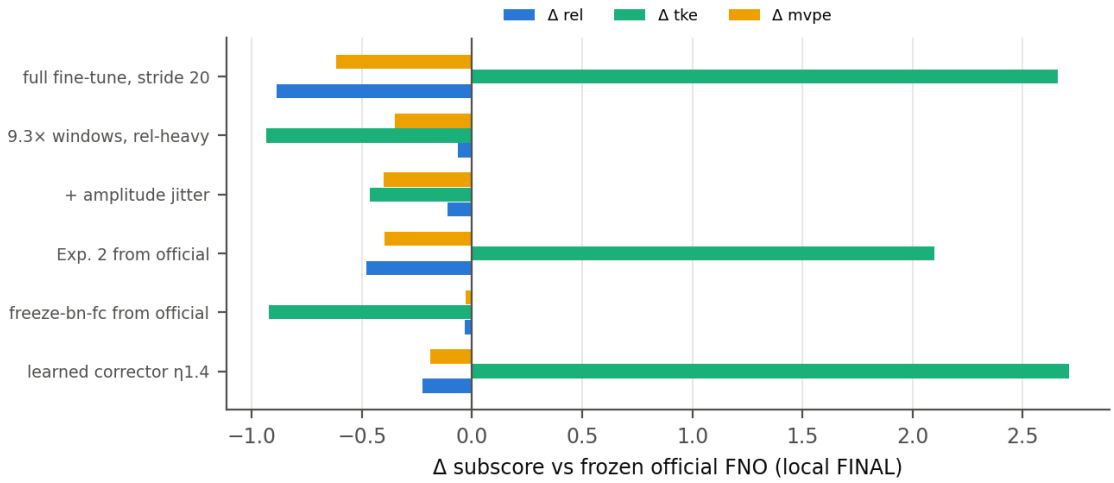


Figure 25. Our fine-tunes of the official FNO and the learned corrector, against the frozen official FNO on local FINAL. Every configuration that buys tke sells rel and mvpe.

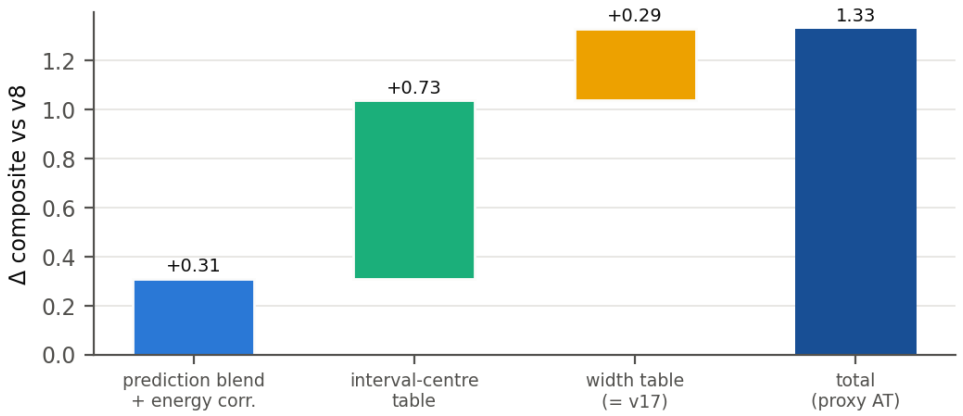


Figure 26. How v17's gain over v8 builds up under the live-calibrated proxy: prediction blend with energy correction, then the interval-centre table, then the width table.

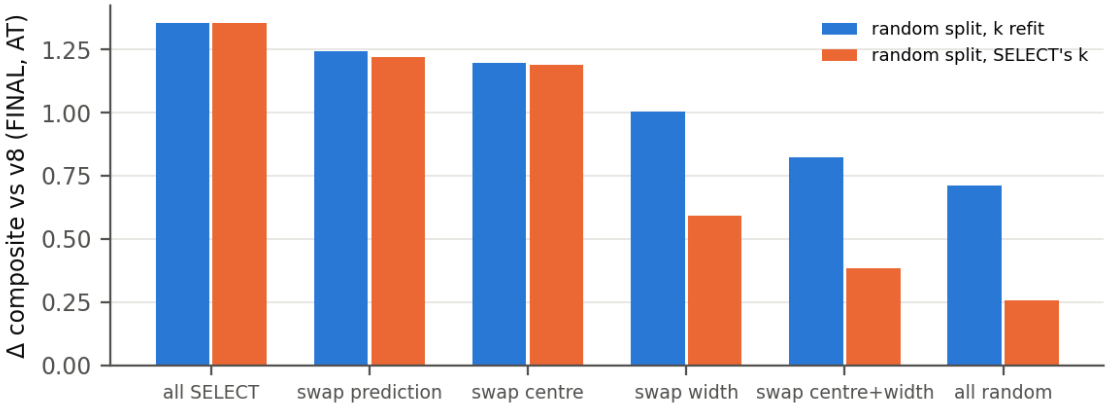


Figure 27. Swap ablation: v17's tables fitted on SELECT, with one component at a time replaced by its randomly-fitted counterpart. The width table carries most of the dependence on the fitting split.

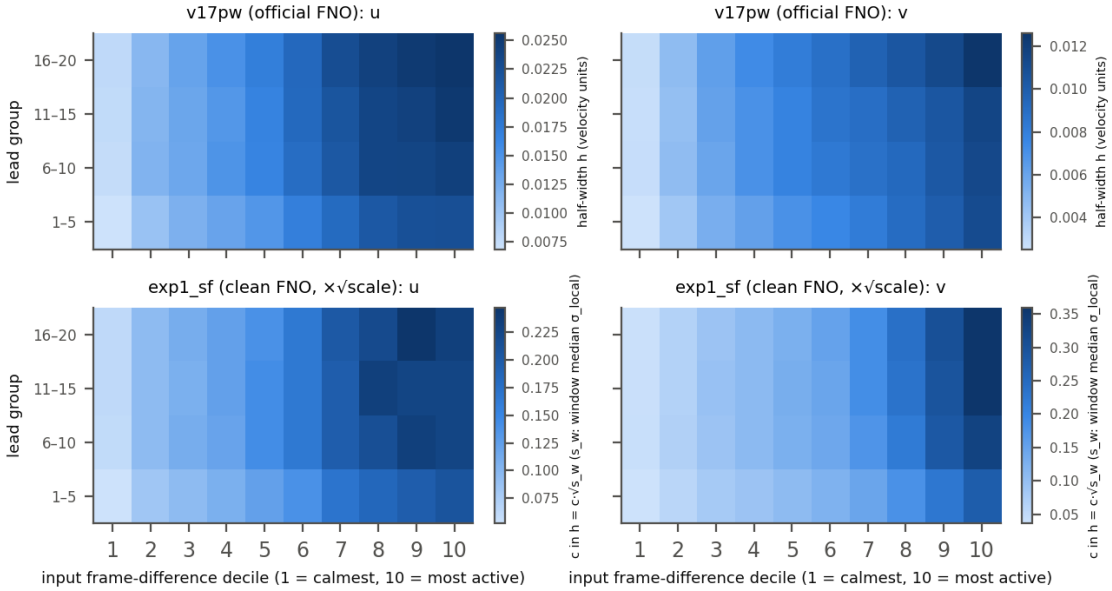


Figure 28. The fitted interval half-width tables of our two final builds, by lead group and input frame-difference decile. Widths grow with input activity and lead; exp1_sf's are scaled per window.

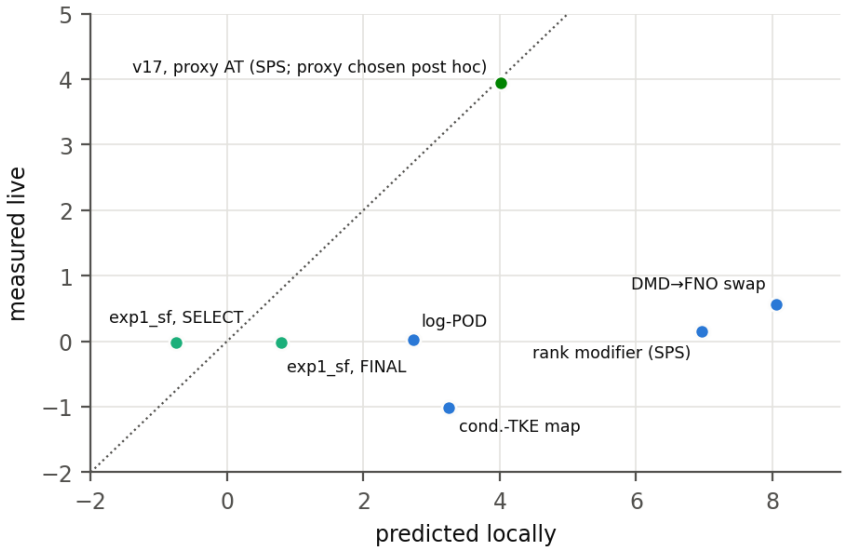


Figure 29. Local prediction against live result for every change we could check. The dotted line is perfect transfer. Early changes validated on possibly-memorised data fell far below it. v17's point uses proxy AT, one of four proxies computed before submission (k_T fitted to v8's live band factor); AT was identified as the closest only after the live result. Our clean model misses it by 0.73 (SELECT predicted −0.75, live −0.02) and 0.82 (FINAL predicted +0.80): smaller misses than the early changes, in both directions.

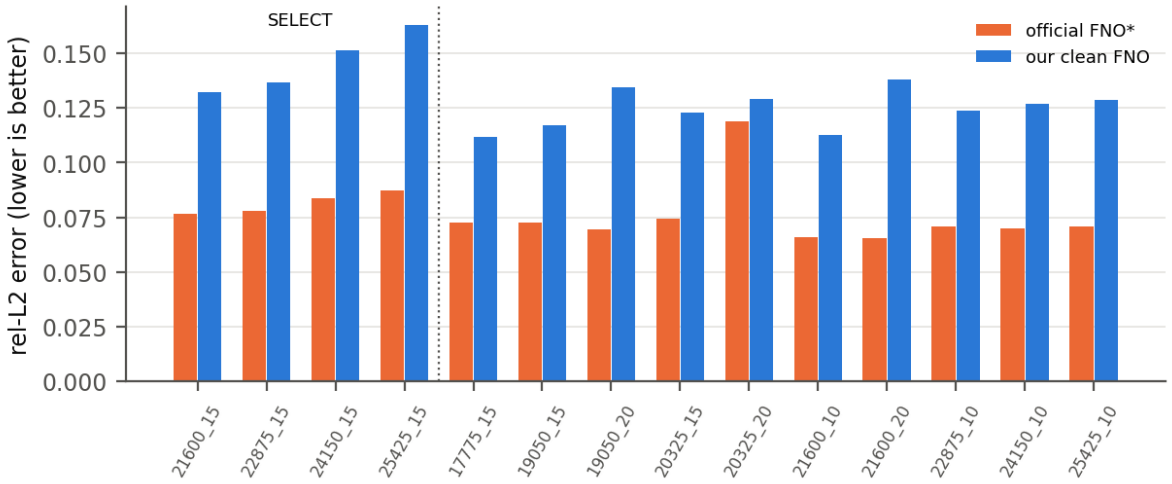


Figure 30. Relative L2 error per trajectory of our clean FNO and the official FNO on SELECT (left of the dotted line) and FINAL. Our model never saw these trajectories; the official one very likely did.

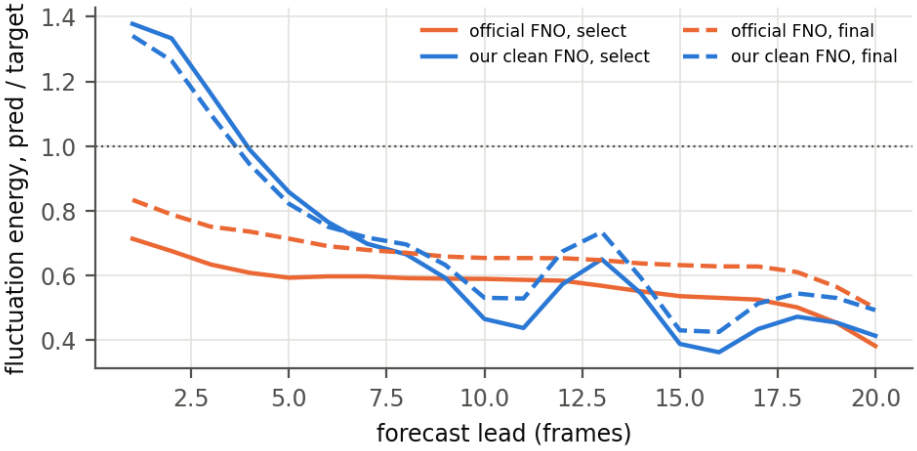


Figure 31. Fluctuation energy of the forecast relative to the target, by lead. Our clean FNO over-shoots at short leads (up to 1.38× at lead 1) and falls to the official model's level after lead 6; both lose energy at long leads, the source of the tke deficit.

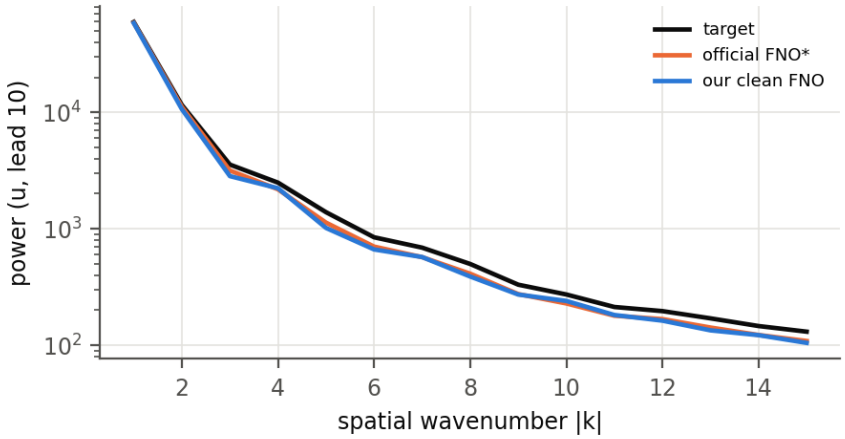


Figure 32. Spatial power spectrum of u at lead 10 on SELECT. Both models under-represent fine scales by a similar amount; the spectra are nearly identical.

12 Limitations

(1) All tables were fitted on 4 trajectories near the evaluation condition. Between 23% and 119% of the local gain on FINAL depends on regime match (13 random splits, median 62%; §7), and the widths are absolute, so performance at unseen AoA/Re is uncertain; Fig. 15 measures how the tables transfer to a held-out angle or Reynolds band, with the forecaster held fixed. (2) The proxies rest on one post-hoc agreement. (3) v17 uses no simulation data at inference; its sim-to-real content is the official checkpoint's. exp1_sf's sim-to-real content is our own fine-tune of the simulation-only FNO (§10), submitted live once: 79.238, 0.023 below v17, after SELECT had predicted −0.75. (4) Frame-correlated PIV error could favour persistence. Calm-pixel window-mean error (0.00295) is roughly twice our noise-floor estimate for freestream u (≤ 0.0016), so part of the calm-region gain may be measurement structure rather than flow. (5) Our best score (79.261) is 2.89 below the 1st-place benchmark (82.151). (6) Proxies A/AT are anchored on the official FNO's residual; cross-model comparisons under them (CNO, ensembles) are unvalidated, and live sampling uncertainty over ~90 windows is unquantified. The √-width choice was also cross-fitted (SELECT −0.83 when fitted on FINAL), not only fitted in-sample (−0.75).

13 Data provenance and compliance disclosures

Only the official release was used. We trained on *train_real* (with *7575_0* excluded) and used the official *sim_real_ft* FNO. The *sim_fno*/*sim_cno* checkpoints served as fine-tuning starts (§4). No external data, externally pretrained weights or generated data were used. All fitted quantities come from released training windows.

Disclosure 1: cross-window rank statistics. From v4 onward, our submissions computed the quantile edges of their rank lookups (the interval-shape modifier and, in v17, the σ_{local} sextiles and frame-difference deciles) over all windows in each predict() call. That is five windows, padded by repeating the last. Each window's prediction and bounds therefore depended slightly on other evaluation windows. The harness also feeds window k's target frames as window k+1's input, so this dependence can reach the answered window's own target. This was not intended, and the effect is a shift of bin edges. The Rules prohibit using information from other evaluation windows, and this falls within that. We rebuilt the method with every rank statistic computed from the answered window alone (v17pw, v19pw; tables refitted with per-window bins). Under the four proxies, the change moves the local composite by +0.002 to +0.006, so the pooling carried no benefit. Each per-window build must pass an independence test before submission: every window's prediction and bounds must be identical, to 10^{-6}, whether it is predicted alone, in any batch, shuffled, padded or duplicated.

Disclosure 2: logged evaluation-input statistics. Several diagnostic submissions wrote aggregate statistics of the evaluation *inputs* to the participant log: means, spectra, autocorrelations and mask sizes. They never included targets. We used them to characterise the live flow condition, and that characterisation chose our local hold-out, the SELECT split and the proxy calibration. The current Rules prohibit encoding evaluation data into logs. We have not been able to confirm whether this applied to aggregate input statistics at the time; no ruling from the organisers had been obtained at the time of deposit. Section 7 measures what this choice was worth locally: 0.65–1.10 of v17's 1.36 proxy composite gain (48–81%) on the first random split, and 0.31–1.62 (23%–119%) over 13 splits. Because FINAL was chosen by the same

fingerprint, that range bounds the value of regime match to FINAL rather than of the logged information as such. The logged values are not published in this report.

Disclosure 3: harness structure. We noticed that consecutive evaluation windows overlap (window k's target is window k+1's input). We did not deliberately exploit it. Disclosure 1 describes an unintended channel through which it could have acted.

use	what information	status
A. Model training (weights or table values)	released *train_real* windows only; official checkpoints as starting points	no evaluation data of any kind
B. Offline model selection	which training trajectories formed SELECT/FINAL; the proxy scale k	chosen using aggregate statistics of logged evaluation *inputs* (Disclosure 2) and our own live scores; regime-match effect on FINAL: 0.65–1.10 of 1.36 proxy composite (48–81%) on the first split; 0.31–1.62 (23%–119%) over 13 splits (K9)
C. Logged by diagnostic submissions	means, spectra, autocorrelations, mask sizes of evaluation inputs; timings	never targets; never read back at inference
D. At inference (live builds v4–v17)	rank-bin edges pooled over the 5 windows of a call (Disclosure 1)	unintended; effect ≤ 0.006 composite locally; removed in v17pw/v19pw
E. After submissions, for analysis only	own live subscores; public leaderboard subscores of all teams	public or our own results; used for §3 and proxy calibration

Table 10. Where each kind of information entered the pipeline.

14 Reproducibility

step	file / command
inference packages	submissions/v17/submission_blend_v17.zip (live); per-window rebuilds in submissions/v17_per_window, submissions/v19_per_window
local caches	experiments/interval_recipes/cache_preds.py → cache_{select,final}_s20.npz (X, Y, P); random split: experiments/robustness/cache_rand.py
splits	data/splits/holdout_v12_3.json (SELECT 4, FINAL 10, train 67), data/splits/train_pool_v12_3.json, experiments/robustness/rand_split.json
prediction / centre / width fits	experiments/estimator_blend/fit.py, fit_band.py, controls2.py → pickles; make_tables.py writes _blend_tables.py
energy correction	experiments/tke_energy/eval_rescale.py (gain_input, γ = 0.5 → exponent 1/4, cap 1.6)
ridge forecaster (v19)	experiments/v19_ridge_partner/accum.py 4; gen_L.py 10 5 1e-4 sxb 1e-3 sxb3; blend19.py
fine-tuning	src/training/finetune_fno_kaggle.py (Kaggle T4; flags --freeze-bn-fc, --spectral-aug, --unfreeze-extra, --pool-file)
analyses in this report	experiments/robustness/*.py; report/data/weights_bootstrap.json; report/data/experiments.json (full register) and experiments_public.json (Appendices B–C, plus the submission records summarised in Appendix A)
gates	src/evaluation/preflight.py ; experiments/estimator_blend/verify_torch.py; per-window independence test (experiments/per_window_rebuild)
figures and report	report/scripts/build_all.py (runs every figure script, then writes this report)

Table 11. Reproduction map. All post-processing is lookup tables fitted in minutes on a CPU; fine-tuning used a Kaggle T4.

Deposit manifest. Code, fitted tables, experiment logs and the scripts that build every figure and this report are in the Git repository that accompanies this Zenodo record (the commit is named on the record page). Our own checkpoints and the exp1_sf package (over 100 MB) are attached to the Zenodo record where the benchmark licence permits. Packages that embed the organisers' official checkpoint (v17, v17pw, v19pw) are identified by hash only; the organisers' starting kit, datasets and official checkpoints are not redistributed, and the kit's instructions retrieve them. Table 12 lists the artefacts behind the headline numbers; full 64-digit hashes are in MANIFEST.sha256 at the repository root (check with *sha256sum -c MANIFEST.sha256*). This report does not reproduce the logged evaluation-input values, but some repository files record them: the code and logs of the diagnostic submissions and of the analyses that used them (Disclosure 2; docs/CAMPAIGN_REPORT.md §5 and §13). The repository stays private until the organisers rule; it will then be published either complete or with those files redacted. The repository URL and the DOI are on the Zenodo record page. Rebuild figures and report with *python report/scripts/build_all.py*.

artefact	what it is	size	SHA-256 (first 16 hex)
checkpoints/exp1_67_best.pth	Run 1 clean sim→real FNO checkpoint (exp1_sf model)	402.8 MB	554a6230d2d3ddf9
checkpoints/fno_finetuned_best.pth	Exp. 2 fine-tune of the official FNO	402.8 MB	cd75a4736d7b0f7b
submissions/v17/submission_blend_v17.zip	v17, live 79.261 (pooled bins)	186.3 MB	5dcb0e1703cd1079
submissions/exp1_sf/submission_exp1_sf.zip	exp1_sf, live 79.238 (per-window)	186.3 MB	811e9cf97ca1a5a7
submissions/v17_per_window/submission_v17pw.zip	v17 per-window rebuild (not scored live)	186.3 MB	fa475e6495a5f039
submissions/v19_per_window/submission_v19pw.zip	v19 per-window rebuild (not scored live)	194.2 MB	2fba0cf43a0ab7ef
submissions/v17_per_window/_blend_tables.py	fitted tables, v17pw	8.3 kB	fc9fa897b95bf2c4
submissions/exp1_sf/_blend_tables.py	fitted tables, exp1_sf	7.5 kB	8f36bb7573906b54
submissions/exp1_sf/submission.py	exp1_sf inference code	34.2 kB	dc66377a94dd529b
data/splits/holdout_v12_3.json	SELECT / FINAL split	1.5 kB	7065667262c5e742
data/splits/train_pool_v12_3.json	67-trajectory training pool	1.0 kB	9d88cc753e67716f
src/training/finetune_fno_kaggle.py	fine-tuning script (all runs)	65.7 kB	bc6294ac4d0d0662

Table 12. Deposit manifest: the artefacts behind the reported live numbers, with sizes and hashes.

Acknowledgements

We thank the RealPDE organisers for the benchmark, the public leaderboard API, and their quick response to the data-duplication report.

Appendix A. Submission ledger

build	change	live final	local preflight (rel/tke/mvpe/SPS)
S1–S3	numpy DMD + interval scale bracket	76.68 / — / 77.01	—
S4	+ log-POD denoise, lazy decode	76.49	—
S5	official FNO, no intervals (±5% default)	72.65	—
S6	FNO + corrector η 0.4 + intervals	77.90	—
v4 (S7)	corrector η 1.4 + rank modifier	78.03	—
v5 (S8)	+ manual CUDA graphs	0 (import crash)	—
v6 (S9)	×0.85 widths, fp16 bounds, modifier off	77.75	—
v7	v8 path + η 1.4, multi-stage loader	not recorded	—
v8	official FNO + heteroscedastic intervals + modifier	78.141	96.06 / 80.29 / 97.20 / —
v9	v8 + target-free diagnostics	78.124	—
v10	v8 + input diagnostics (host syncs)	not recorded (time 83.09)	96.06 / 80.29 / 97.20 / 42.91
v11	flat intervals, modifier off	not recorded	96.06 / 80.29 / 97.20 / 39.41
v12	v8 with modifier off	77.899	96.06 / 80.29 / 97.20 / 35.43
v13	flat intervals + modifier	not recorded	96.06 / 80.29 / 97.20 / 31.54
v14	fine-tuned FNO swapped into v8 (planned)	never built	—
v15	v8 + AOT-compile / requests probe	not recorded	—
v16	v8 + energy correction	not submitted	same pool vs v8: −0.01 / +2.07 / 0 / +0.55
v17	v16 + blend + interval tables	**79.261**	96.10 / 82.54 / 97.20 / 44.08
v18	v17, unconstrained blend	not submitted	96.08 / 82.38 / 97.26 / 44.05
v18ft	v17 post-processing on fine-tuned FNO	not submitted	95.46 / 82.12 / 96.70 / 41.42
v19	v17 + ridge forecaster partner	not submitted	96.14 / 82.73 / 97.20 / 44.54
v17pw / v19pw	per-window (compliant) rebuilds of v17 / v19	not submitted	96.10 / 82.55 / 97.20 / 44.08 ; 96.14 / 82.74 / 97.20 / 44.55
exp1	Run 1 clean FNO + honest interval tables	not submitted	93.13 / 78.66 / 94.79 / 34.31 (unseen pair)
exp1_sf	exp1 with √-scaled widths	79.238	honest FINAL 94.16 / 79.18 / 95.50 / 38.15

Table A1. All builds. Local preflight scores are on the memorised FINAL pair and only compare builds with each other.

Appendix B. Every experiment: question, exact procedure, data, result, reading

Several records below use the word “memorised” as shorthand; read them with the evidence grading of Table 2 (an inference, not an established fact). Experiments change something and compare it with a baseline. The twelve that shaped the campaign have full cards in §8, and every register experiment appears below, except EM09 (a leaderboard-wide analysis of other teams, omitted because this report uses no other-team result except the 1st-place score), the clean sim-to-real work of §10, the K9 swap ablation, the K10 analysis and the per-window rebuild (see results/live_logs/, experiments/clean_fno_regimes, experiments/cno_check, experiments/ensemble_check, experiments/robustness, experiments/per_window_rebuild). Results are tagged live, local (memorised split), local-honest or proxy. Numbers were checked against the files in the last row of each block. Details not recorded in any source are marked as such.

B.1 Intervals and SPS

BS01 Approximate SPS factorisation and the reversed-hazard optimum [adopted]

Question / hypothesis	SPS can be split into an accuracy part and a band part that can be optimized separately.
What was done	Derived from scoring.py: all three SPS branches share the inside mask and exp(-width) term.
Data / split	analytic (scoring.py v9)
Result	sps = 100 * Wbar * T; Wbar = 0.5(1-pm_dm)+0.3(1-pm_tke)+0.2(1-pm_mvpe); T = mean exp(-2h/sigma)*1{|e|<=h}, sigma 0.0563870. Optimum f(h)/F(h) = 2/sigma = 35.469; measured optimal coverage 0.70-0.80. Live Wbar 0.6784 (v8), live T 0.4914.
Reading	Over-coverage is punished exponentially and under-coverage only linearly.
Scripts / source	docs/CAMPAIGN_REPORT.md §2; docs/literature/interval_methods.md §0

BS02 Rank-based per-element band modifier (sigma_local sextile, within-call rank) [adopted]	
Question / hypothesis	Narrowing bands on high-sigma elements and widening on calm ones raises T; ranks survive the live scale shift.
What was done	Multiplier table per within-call sigma_local sextile (u 1.5..0.25, v 0.9..0.25) fitted on SELECT, shipped in v4 and kept through v17.
Data / split	SELECT fit / FINAL eval (memorised); live
Result	local: +6.96 sps on FINAL (CAMPAIGN_REPORT §6.1). live: S6->S7 sps 32.397 -> 32.549 (+0.152, 2.2% transfer; confounded with eta 0.4->1.4). v12 (modifier off, else v8) live final 77.899 vs v8 78.141.
Reading	A one-feature shape effect; the direction transferred but only a few percent of the local size.
Scripts / source	submission_fno_boosted_v4.zip, submission_fno_heteronomod_v12.zip · docs/CAMPAIGN_REPORT.md §3, §6.1, §9; submission_fno_boosted_v4.zip submission.py; submission_fno_flatbandmod_v13.zip submission.py (v12 live 77.899)
BS03 Band frontier, feature ceilings and band-shape closure [closed]	
Question / hypothesis	Extra features (|prediction|, spectral energy ratio, temporal disagreement) or learned predictors add band-shape T beyond sigma_local.
What was done	On 84 held-out windows (6,144,971 elements) measured T for flat, 1-D and joint lookups; nested TRAIN/SELECT/FINAL over 8 AoA-15 trajectories with within-call ranks; synthetic payoff curve; plus variants (fixed width, bias shift, per-group, per-lead, James-Stein, per-AoA, ACI, disagreement probe, learned predictor).
Data / split	84 held-out windows; 8 AoA-15 trajectories (memorised)
Result	local T: shipped 0.4653, best flat h=0.009 0.5217, joint 3-feature ceiling 0.5663, oracle 0.7851. Nested FINAL: sigma_local rank alone 0.5552 (+19.50%), 3-feature 0.5576 (+0.43% more). Fixed width -2.76; per-group +0.18; per-lead +0.13; disagreement probe +0.0101 T vs -1.47 time; learned predictor Spearman 0.6534 vs floor 0.75 (fail). Payoff: Spearman 0.52->0.90 buys T 0.5596->0.6421 (synthetic upper bound).
Reading	The gain is one feature already shipped; real features deliver 0.53-0.93x of the synthetic payoff.
Scripts / source	docs/CAMPAIGN_REPORT.md §6.1, §9 'Band shape', §12.7
BS04 Claim that bands sit at their analytic optimum (max +0.15 composite) [closed]	
Question / hypothesis	Live T=0.4914 is already at the reversed-hazard optimum for any plausible error scale, so SPS has no headroom.
What was done	Analytic reversed-hazard argument (true_gradients.py).
Data / split	analytic
Result	Stated: absolute maximum any band scheme can add is +0.15 composite; the SPS band track was closed. Contradicted later by our own v17, which gained live SPS +3.949 (band factor 0.491 -> 0.545) from band centre/width changes.
Reading	The argument assumed bands centred on the prediction with a sigma_local-type width; moving the band centre and using other width features was outside it.
Scripts / source	docs/CAMPAIGN_REPORT.md §14.1, §14.4; report/data/lb_analysis.json; results/live_logs/live_v17.txt
BS05 Band recipe search under a live-calibrated proxy (experiments/interval_recipes) [adopted]	
Question / hypothesis	Band SHAPE is the largest remaining lever (our band factor T was 0.491, and 1st place's SPS is 44.97 against our 33.34) and needs no retraining.
What was done	Dumped per-element |e| for the 14-trajectory holdout under proxy targets Y'=P+k(Y-P); fitted width recipes (flat, 1-D lookups, ridge log-feature score, Laplace formula, GBM, L1 centre shift) on SELECT and scored once on FINAL with the official scorer. Also experiments/piv_noise residual-feature ranking.
Data / split	holdout_v12_3 SELECT (fit) / FINAL (eval), stride 20, proxy targets
Result	proxy T/T (k 1.394/1.447) FINAL T: v8 0.4914, flat 0.5033, 1-D p1 (frame-diff RMS) lookup 0.5178, ridge log-feature 0.5222, + L1 centre shift 0.5240; official sps 35.38 -> 37.72. GBM: u lookup T 0.3521 vs Laplace 0.3480; top gain p1 (0.556 u, 0.524 v). experiments/piv_noise SELECT Spearman with |e|: hf_d2 0.615, sigma_max5 0.571, sigma_local 0.526.
Reading	One-step frame-difference RMS ranks element error better than sigma_local, and the width curve is concave, so a lookup beats A+K*sigma.
Scripts / source	experiments/interval_recipes/recipes.py, experiments/interval_recipes/gbm.py, experiments/interval_recipes/centre.py, experiments/interval_recipes/final_official.py, experiments/interval_recipes/export_recipe.py, experiments/interval_recipes/explore.py, experiments/interval_recipes/check_scale.py, experiments/piv_noise/resid.py, experiments/piv_noise/tgain.py, experiments/piv_noise/recenter.py, experiments/piv_noise/group_cal.py · experiments/interval_recipes/recipes_out.txt, gbm_rel_out.txt, centre_out.txt, final_official_out.txt; experiments/piv_noise/resid_select.txt
BS06 Literature scan of interval methods for the T objective [adopted]	
Question / hypothesis	Established interval-prediction methods map onto the gaps in the v8 band.
What was done	Desk review of heteroscedastic heads, RQR/QD/Tube losses, ACI, Mondrian/Kandinsky conformal, CQR, UQ neural operators; derived Laplace/Gaussian optimal h*(s).
Data / split	analytic/literature
Result	h*(s) is concave and saturates at sigma/2 ~0.028 (Gaussian s=0.005 -> h* 0.0089, T* 0.68; s=0.05 -> 0.0258, 0.16); linear A+K*s is structurally wrong; T* is convex in s so error-predictor resolution matters most. Coverage-targeted conformal and cross-window methods rejected.
Reading	The optimal interval per element is a highest-density interval around the residual mode, not a fixed-coverage band.
Scripts / source	docs/literature/interval_methods.md · docs/literature/interval_methods.md §0 and §1-6

B.2 TKE

TK01 TKE denoising family (14 variants) including log-POD + TKE target ceiling from split-half disagreement [closed]	
Question / hypothesis	Denoising the variance field of the DMD forecast raises the tke subscore.
What was done	log-POD rank-4 variance denoise, reliability-weighted POD, vash priors, SPOD thresholding, velocity truncation, per-pixel n_eff shrinkage, tensor invariants, ensemble-Kalman inversion; best shipped live as submission S4. Also: Compared split-half variance estimates and a long-window estimate of the same field.
Data / split	regime-matched pool (memorised), nested TRAIN/SELECT/FINAL
Result	local: log-POD +2.31 (best), reliability-POD +2.1, vash +0.72, tensor invariants -6.62, others negative. live (S4): final 76.4927 vs 77.0117 incumbent; tke gain +0.03 of +2.73 local (1% transfer); lazy decode in predict cost -0.552 time. | Split-half rel-L2 0.84-0.89; vs long-window estimate ~0.50; a predictor knowing the converged field exactly scores 79-81.
Reading	Local gain was measured on memorised data; the 2-D PIV slice is not divergence-free (RMS div 1.49x terms), which also sinks invariant-based priors. The target is itself a 20-sample variance estimate, so it carries irreducible sampling noise.
Scripts / source	docs/CAMPAIGN_REPORT.md §3, §6.2, §6.5 transfer table; results/closed_directions.json C09; docs/CAMPAIGN_REPORT.md §6.2

TK03 Diagnosis of the stock FNO tke deficit (amplitude vs phase) [adopted]	
Question / hypothesis	The tke error comes mostly from missing amplitude, not wrong position or phase.
What was done	Energy ratios, spatial KE correlation, oracle per-window scalar, lead-wise fluctuation ratios, regional ratios, masked-pixel share, and input autocorrelation/periodic persistence on the holdout caches.
Data / split	holdout_v12_3 SELECT 168 / FINAL 420 windows (memorised)
Result	local FINAL: energy ratio pred/target median 0.622 (SELECT 0.569); spatial corr 0.903; oracle scalar tke 83.84 vs 81.04 (+2.8); input energy/target 0.997; masked-pixel share of tke sq-error 0.0%; lead fluct ratio 0.83 -> 0.50. Input autocorrelation min at lag 7 (-0.59), ~0 from lag 13; periodic persistence has no skill beyond lead 0-2.
Reading	MSE training damps fluctuation amplitude while roughly keeping phase; the stationary input window gives a target-free amplitude reference.
Scripts / source	experiments/tke_energy/diag_tke.py, experiments/tke_energy/phase_skill.py · experiments/tke_energy/diag_tke_out.txt; experiments/tke_energy/phase_skill_out.txt; experiments/tke_energy/tke_analysis.md §1-2

TK04 Mean-preserving fluctuation gain with bands kept on the stock centre (pre-registered) [live-shipped]	
Question / hypothesis	Scaling each window's fluctuation about its own mean buys tke without moving mvpe, and leaving the bands on the stock prediction avoids the corrector's SPS loss.
What was done	Global a (G), input-anchored a_w=clip((KE_X/KE_P)^(gamma/2),1,1.6) (I), lead/regional variants; selected on SELECT, FINAL touched once; primary G a=1.15 pre-registered.
Data / split	holdout_v12_3 SELECT/FINAL (memorised)
Result	local FINAL: G a=1.15 band on stock P rel -0.069 tke +1.376 mvpe 0 sps +0.275 comp +0.152; I gamma=0.5 rel -0.053 tke +1.619 sps +0.391 comp +0.209; same G with band recentred sps -0.687 comp -0.082. Per trajectory: helps 8/10 (vr 0.76-0.81), hurts AoA-20 pair (vr 0.92, G dtke -1.6/-1.8). Shipped as v16 (not submitted) and inside v17.
Reading	mvpe uses time means so it is invariant; the boost helps most where the input variance ratio is low.
Scripts / source	experiments/tke_energy/eval_rescale.py, experiments/tke_energy/per_traj.py, experiments/tke_energy/PREREG.txt · experiments/tke_energy/eval_rescale_final_out.txt, eval_rescale_select_out.txt, per_traj_out.txt, PREREG.txt; experiments/tke_energy/tke_analysis.md §3-5

B.3 Models

MA01 numpy DMD rank-6 forecaster with variance top-up [closed]	
Question / hypothesis	A small, fully specified linear dynamics model is competitive without learned weights.
What was done	Rank-6 DMD extrapolation of the input window plus variance top-up; 13 KB; later swept RANK/RHO/TKE_GAIN in combo_symbolic.
Data / split	live; cluster-1 FIT pool
Result	live S3: rel 92.501 tke 75.129 mvpe 92.629 time 90.866 sps 31.879 final 77.0117 (7.37 ms/sample). Local sweep: best (rank 4, rho 0.95, gain 0) worth +0.515 standalone w over shipped (6, 0.95, 0.8). Superseded by the FNO.
Reading	DMD errors are much larger than the FNO's even though they are only +0.53-0.56 correlated.
Scripts / source	submission.py (DMD rollback source), experiments/early_diagnostics/combo_symbolic.py · docs/CAMPAIGN_REPORT.md §3 row 3, §6.3; results/logs/combo_symbolic_out.txt STAGE 1

MA02 Adopt the organizers' sim_real_ft FNO checkpoint [adopted]	
Question / hypothesis	The organizer FNO beats the DMD incumbent.
What was done	Loaded sim_real_fno_fp16.pth at import (untimed), scored locally and live.
Data / split	matched pool (memorised); live
Result	local FINAL: rel 96.06 tke 80.29 mvpe 97.20 (boosted_scores stock 96.064/80.287/97.202). live (v8): rel 94.168 tke 74.026 mvpe 92.836. Model swap local +8.05 vs live +0.561 (7% transfer).
Reading	It is the strongest available model, but local numbers were training scores (see EM04).
Scripts / source	src/evaluation/build_fno_zip.py, src/evaluation/score_holdout_v12_3.py · results/boosted_scores.json FINAL.stock; docs/CAMPAIGN_REPORT.md §6.3, §6.5 transfer table

MA03 Organizer CNO and Transolver checkpoints (and a CNO placeholder build) + Own models: from-scratch invariant model and PCA+ridge CNO distillation [closed]	
Question / hypothesis	A different organizer architecture beats the FNO on this regime.
What was done	Scored stock CNO/Transolver; built a CNO submission with a vendored requests stub (never shipped). Also: Trained a 9.45M-parameter invariant model from scratch; distilled CNO into PCA+ridge.
Data / split	local SELECT/FINAL (memorised); holdout_v12_3
Result	local: CNO rel 95.95 tke 78.77 mvpe 96.50 (loses all three to FNO); Transolver tke 70.69. Earlier '+3.60 rel / +1.73 mvpe' was measured against the DMD, not the FNO. Frozen sim_cno on holdout final-10: 78.145/67.145/74.743. not submitted live (placeholder never submitted). | local: from-scratch 266.0 vs FNO 273.6 (sum of rel+tke+mvpe); distillation R^2 0.42, 11% of the MVPE gain.
Reading	The CNO loses on every metric to the FNO; its apparent win was against the wrong baseline. Both are well below the organizer FNO locally.
Scripts / source	src/evaluation/_vendor_requests_stub.py, src/evaluation/build_cno_zip.py, src/evaluation/score_cno_ckpt.py, submission_cno_placeholder.zip · docs/CAMPAIGN_REPORT.md §6.3; results/holdout_v12_3_simcno_frozen_scores.json; submission_cno_placeholder.zip submission.py docstring; docs/CAMPAIGN_REPORT.md §6.3

MA05 FFT padding retune to 7-smooth sizes [closed]	
Question / hypothesis	Changing domain padding to FFT-friendly sizes speeds up inference at no accuracy cost.
What was done	Rescored the FNO at padding 4, 7, 8, 10, 12 on SELECT (84 windows).
Data / split	SELECT 84 windows
Result	local delta sum3: pad 4 -7.453, 7 -3.965, 8 (7-smooth) -9.780, 10 -18.167, 12 -22.638.
Reading	Padded length changes which physical frequency each retained mode represents; the weights are tied to padding 6.
Scripts / source	docs/CAMPAIGN_REPORT.md §6.4

MA06 FNO architecture and storage budget [closed]	
Question / hypothesis	A second ensemble member could fit in the submission.
What was done	Counted parameters from the checkpoint.
Data / split	checkpoint
Result	50,357,955 params, 99.9% spectral convs; modes (4,12,16), width 64, 4 layers, padding 6 (FFT 26,38,70); no dropout. fp16 pack 201,396,349 bytes vs 256 MB cap.
Reading	Storage, not time, rules out deep ensembles.
Scripts / source	docs/CAMPAIGN_REPORT.md §6.3, §4
MA07 Off-the-shelf RealPDEBench UNet and DPOT-S (pilot) [inconclusive]	
Question / hypothesis	Upstream foil checkpoints (UNet reported ~30% better than FNO in the paper) transfer to this regime.
What was done	Loaded vendored UNet and DPOT-S foil fine-tune checkpoints, normalised with kit statistics, scored on SELECT.
Data / split	holdout_v12_3 SELECT (4 traj)
Result	local SELECT: FNO 96.088/79.915/97.379; UNet 89.019/70.818/86.112; DPOT-S 88.818/70.072/84.364; frozen sim_cno 77.531/67.077/73.807. A DPOT run without normalisation gave rel 64.0 (bug).
Reading	Both were trained at 128x64 and fed 32x64; no resolution-matched fine-tune was run.
Scripts / source	experiments/early_diagnostics/pilot_unet_select.py, experiments/early_diagnostics/pilot_dpot_test.py, src/evaluation/score_unet_ckpt.py, src/evaluation/rpdebench_unet.py · docs/CAMPAIGN_REPORT.md §15.4; experiments/early_diagnostics/pilot_dpot_test.py header
MA08 Honest ridge linear forecaster L (v19 component) [adopted]	
Question / hypothesis	A local linear operator trained only on the 67 non-holdout trajectories forecasts calm regions better than window estimators, with unbiased local scores.
What was done	Ridge on 10 input frames x 5x5 neighbourhood per sigma sextile x 4x4 spatial block; sweeps over m, k, lambda.
Data / split	trained on train_pool_v12_3 (67 traj); scored on holdout SELECT/FINAL
Result	local FINAL (honest, never saw holdout): best rel-L2 0.1260 (m10 k5 lam1e-4), calm-pixel MAE 0.00283 vs window mean 0.00295 and last frame 0.00328; stock FNO P rel 0.0751 (memorised). CPU cost 9.6 ms/sample; GPU cost unmeasured.
Reading	Used only as a constrained blend partner, since standalone it is far worse than the FNO.
Scripts / source	experiments/v19_ridge_partner/accum.py, experiments/v19_ridge_partner/accum2.py, experiments/v19_ridge_partner/ridge.py, experiments/v19_ridge_partner/gen_L.py, experiments/v19_ridge_partner/standalone.py · experiments/v19_ridge_partner/standalone_out.txt; tools/decisions/d006_resume_v19.json

B.4 Fine-tuning (sim-to-real)

FT01 First FNO fine-tunes on train_real (stride 20, sensitivity weights) [closed]	
Question / hypothesis	Fine-tuning the organizer FNO on the eval-like regime improves all subscores.
What was done	Full-model fine-tune on 3,362 windows (stride 20), loss weighted by metric sensitivity; second run with w_rel 0.70.
Data / split	SELECT/FINAL quad (memorised)
Result	local FINAL (tuned vs stock): rel 95.175 vs 96.064 (-0.889), tke 82.946 vs 80.287 (+2.66), mvpe 96.584 vs 97.202 (-0.618), sps 33.420 vs 35.432 (-2.01). Run 2 (w_rel 0.70): rel -0.269 tke +2.245 mvpe -0.242.
Reading	The only productive direction from the converged checkpoint is an output-variance rescale that buys tke and sells rel/mvpe.
Scripts / source	src/training/finetune_fno_kaggle.py, src/evaluation/score_finetuned_ckpt.py · results/finetuned_scores.json FINAL; src/training/boost_operator_kaggle.py docstring; docs/CAMPAIGN_REPORT.md §6.3, §9
FT02 Re-fine-tunes with 9.3x more windows, tke-suppressed loss and amplitude/noise augmentation [closed]	
Question / hypothesis	The first fine-tune was under-determined (3,362 windows); stride 2 plus augmentation fixes it.
What was done	Stride-2 (31,343 windows) runs A (sensitivity weights) and B (0.8/0.0/0.2), 7575_0 excluded, noise_std 0.02; then amp_jitter 0.70 + noise 0.02 with w_tke=0, 12 epochs.
Data / split	SELECT/FINAL quad (memorised)
Result	local FINAL: run B rel -0.063 tke -0.934 mvpe -0.352 (sum -1.350); best SELECT epoch still -1.255 vs no training. amp_jitter: rel -0.111 tke -0.464 mvpe -0.403 (sum -0.979). Run A flat after epoch 0.
Reading	The checkpoint was already organizer-fine-tuned on this data; with the variance-rescale route removed only overfitting remains (train loss -15% while held-out fell).
Scripts / source	src/training/finetune_fno_kaggle.py · docs/CAMPAIGN_REPORT.md §9 'Augmented fine-tune', 'decisive next test'; results/closed_directions.json C01, C10
FT03 Operator-boosting corrector (TinyOperator) on the frozen FNO + Regime-specialist corrector on the live cluster [closed]	
Question / hypothesis	A zero-initialised residual corrector with eta selected on the subscore sum cannot make the base worse.
What was done	668,316-param 2-D spectral corrector fitted to FNO residual; eta grid to 3.0; shipped live at eta 0.4 (S6) and 1.4 (v4, v6); v8 removed it. Also: Corrector trained on 11 of the 19 cluster-1 trajectories, evaluated on 5 untouched holdout trajectories.
Data / split	SELECT/FINAL quad (memorised); live
Result	local FINAL eta1.4 vs stock: rel -0.226 tke +2.711 mvpe -0.190 sps -0.318; eta0.4: rel +0.019 tke +0.852 mvpe +0.006 sps +0.172. live v8 (no corrector) vs v4 (eta1.4): rel +0.148, tke -1.699, mvpe +0.029, sps +0.789, final +0.108. Gate-1 transfer: rel 65.5%, tke 62.7%, mvpe 15.3% (mean 47.8%). | local: +0.178 weighted on holdout, but rel -0.110 (tke weight 0.131 vs rel 0.725 in the loss).
Reading	The corrector moved the time mean and the bands were centred on the corrected field, so SPS lost more than tke gained.
Scripts / source	src/training/boost_operator_kaggle.py, src/training/cluster_specialist_kaggle.py, src/evaluation/score_boosted_ckpt.py · results/boosted_scores.json; docs/CAMPAIGN_REPORT.md §3, §9, §13.4 Gate 1; results/closed_directions.json C02; experiments/tke_energy/tke_analysis.md §4; docs/CAMPAIGN_REPORT.md §12.5

FT05 Linear corrector micro-gate (Gate 3) [inconclusive]	
Question / hypothesis	A closed-form ridge from FNO prediction plus sigma_local, framediff and mask features reduces the residual.
What was done	Streaming ridge (8 features -> residual u,v) on the multivariate live-similarity top-2000 pool, holdout excluded; rerun AoA-15-only with a live-similarity-ranked holdout.
Data / split	1,903 windows / 46 traj train; 35 holdout windows
Result	local: residual 0.01285 -> 0.01281 (+0.325%) on 77,946,880 elements; 'NO SIGNAL'. An AoA-15 rerun was stopped at 256/500 training windows (no result). Gate 2 selection: top-500 windows span 20 trajectories (max share 8.4%).
Reading	First pass showed no linear signal; the rerun that would close the gate has no recorded result.
Scripts / source	experiments/early_diagnostics/linear_micro_gate.py, experiments/early_diagnostics/linear_micro_gate_aoa15.py, src/splits/window_similarity_select.py, src/splits/window_similarity_aoa15.py · results/logs/linear_gate_out.txt; results/logs/gate3_aoa15_out.txt; docs/CAMPAIGN_REPORT.md §13.4 Gates 2-3
FT06 AdaBN test-time BatchNorm statistics [closed]	
Question / hypothesis	BN running statistics pooled over 81 trajectories mis-centre the single live condition.
What was done	Set running_mean/var to None so BN uses batch statistics in eval, batches of 5 as in the harness.
Data / split	held-out trajectories (batches of 5)
Result	local: -0.81 composite, negative on 5/5.
Reading	Live sits inside the training range, so there is no BN-statistic shift to correct.
Scripts / source	experiments/early_diagnostics/adabn_test.py · docs/CAMPAIGN_REPORT.md §14.4; results/closed_directions.json C05
FT07 WiSE-FT weight interpolation sim_fno <-> sim_real_fno [closed]	
Question / hypothesis	Interpolating back toward the sim-pretrained weights recovers robustness under shift.
What was done	theta(alpha) = (1-alpha) theta_sim + alpha theta_sim_real_ft, alpha swept; alpha 1 reproduces the incumbent.
Data / split	held-out trajectories
Result	local: monotonically catastrophic, -7.5 at alpha 0.95; ||theta_ft - theta_sim|| / ||theta_sim|| = 1.094.
Reading	The two checkpoints are not in one basin, so interpolation crosses a high-loss region.
Scripts / source	experiments/early_diagnostics/wiseft_test.py · docs/CAMPAIGN_REPORT.md §14.4; results/closed_directions.json C04
FT08 Experiment 1: clean-start fine-tune from sim_fno on the 67-trajectory pool [closed]	
Question / hypothesis	Training from the sim-only checkpoint on non-holdout data generalises to the honest holdout.
What was done	sim_fno.pth, lr 1e-4, 80 epochs, unfrozen, TRUE_W loss, train_pool_v12_3.
Data / split	train_pool_v12_3 (67) / holdout SELECT 4 / FINAL 10
Result	local FINAL(10): rel 94.266 tke 78.782 mvpe 95.505 (sum 268.554), -5.928 sum vs frozen sim_real_fno holdout (274.482); plateaued at epoch 10.
Reading	Compared against a memorised baseline (EM06), and full unfreezing again traded rel for tke.
Scripts / source	src/training/finetune_fno_kaggle.py, src/splits/build_holdout_v12_3.py · docs/CAMPAIGN_REPORT.md §15.1
FT09 Experiments 2-3: unfrozen fine-tune from sim_real_fno (lr 2e-5) -- the v18ft checkpoint (never submitted) [open]	
Question / hypothesis	A gentle fine-tune of the organizer checkpoint on non-holdout data improves the holdout.
What was done	sim_real_fno.pth, lr 2e-5, 30 epochs, unfrozen; Exp 3 = seed 1, restarts 2. Scored at official stride 20.
Data / split	holdout FINAL 10 (memorised by the stock baseline)
Result	local FINAL raw: ft rel 95.899 tke 83.137 mvpe 96.668 vs stock 96.379/81.039/97.064 (rel -0.480, tke +2.099, mvpe -0.396), TRUE-composite -0.10. Internal stride-2 FINAL said +2.374 sum vs stride-20 +1.223 sum. Packed to fp16 and shipped in v18ft (never submitted).
Reading	Locally a wash with the familiar tke-for-rel trade, but the stock baseline memorised this holdout, so only a live test can judge it.
Scripts / source	src/training/finetune_fno_kaggle.py, src/evaluation/score_finetuned_ckpt.py, src/evaluation/score_boosted_ckpt.py, experiments/v18_refits/cache_ft.py · docs/CAMPAIGN_REPORT.md §15.1; experiments/v18_refits/study_ft_out.txt RAW FINAL lines; src/evaluation/score_boosted_ckpt.py docstring; tools/decisions/d005_final_two_slots.json
FT10 --freeze-bn-fc + spectral augmentation from sim_fno (clean start) [closed]	
Question / hypothesis	Freezing the spectral weights removes the variance-rescale route and leaves a real gradient direction.
What was done	Train only fc0/fc1/fc2/bns.* (spectral convs frozen), spectral input augmentation, 30 epochs, best epoch 21 by SELECT.
Data / split	train_pool_v12_3 / holdout FINAL 10
Result	local FINAL vs frozen sim_fno: rel 79.709 -> 92.462 (+12.753), tke 66.872 -> 71.939 (+5.067), mvpe 75.199 -> 93.527 (+18.327); sum +36.147.
Reading	The first fine-tune to win all three subscores, but it starts too far below the organizer checkpoint to be worth shipping.
Scripts / source	src/training/finetune_fno_kaggle.py · docs/CAMPAIGN_REPORT.md §15.6
FT11 --freeze-bn-fc + spectral augmentation from sim_real_fno [closed]	
Question / hypothesis	The freeze-bn-fc win carries over to the organizer checkpoint.
What was done	Same recipe from sim_real_fno.pth, lr 2e-5, 30 epochs; best epoch 3 by SELECT; FINAL touched once.
Data / split	train_pool_v12_3 / holdout FINAL 10
Result	local FINAL: stock 95.398/77.491/96.184 (sum 269.072) vs ft 95.366/76.568/96.156 (268.090): rel -0.032 tke -0.923 mvpe -0.028. SELECT below stock (267.382) on all 30 epochs.
Reading	fc/bn is the output calibration layer, which the organizers already tuned; with the backbone frozen it can only drift away.
Scripts / source	src/training/finetune_fno_kaggle.py · docs/CAMPAIGN_REPORT.md §15.8; results/closed_directions.json C03

FT12 Spectral input augmentation matched to the live spectrum [closed]	
Question / hypothesis	Boosting input fluctuation spectra toward the live spectrum closes a distribution gap.
What was done	Per radial bin amplitude gain sqrt(power ratio) on inpainted fluctuation fields; verification gate within +/-15% of live; pool of 11 live-similar AoA-15 trajectories; BN/fc-only dry run.
Data / split	live-similar AoA-15 pool (11 traj)
Result	power-vs-amplitude bug caught before training; axes bug fixed; GATE PASS on 473 windows. data/splits/live_pool.json (a different 11-trajectory pool excluding 6 holdout trajectories): gate_pass false. BN/fc dry run: 9,667 of 50,357,955 params trainable, 4,565 train windows.
Reading	EA06 later showed no first-order spectral gap between local and live, so the augmentation had no gap to close.
Scripts / source	src/training/spectral_augment.py, src/training/spectral_gain.py, src/splits/build_live_pool.py, src/training/finetune_bnfc_kaggle.py · docs/CAMPAIGN_REPORT.md §13.6; data/splits/live_pool.json; results/logs/bnfc_dryrun_out.txt; src/training/spectral_augment.py docstring

FT13 CNO-1: clean-start CNO fine-tune on the 67-trajectory pool [open]	
Question / hypothesis	CNO may not hit the FNO's buy-tke/sell-rel wall.
What was done	finetune_cno_kaggle.run_cno1 (sim_cno.pth, 40 epochs, lr 1e-4, stride 2, AMP, batch 16 x accum 8, eval batch 64); five Kaggle upload bundles iterated on OOM fixes.
Data / split	train_pool_v12_3 / holdout (fair baseline: frozen sim_cno final-10 78.145/67.145/74.743)
Result	The run did not complete. FP32 batch 64 and AMP batch 32 OOM on a 14.56 GiB T4.
Reading	Run never completed or its log was never recorded.
Scripts / source	src/training/finetune_cno_kaggle.py, src/training/check_kaggle_uploads.py, src/evaluation/score_cno_ckpt.py, kaggle_upload_cno.zip, kaggle_upload_cno_amp_v2.zip, kaggle_upload_cno_amp_v3_clean.zip, kaggle_upload_cno_amp_v4_batch16.zip, kaggle_upload_cno_amp_v5_batch16_eval64.zip · src/training/finetune_cno_kaggle.py run_cno1 comments; results/holdout_v12_3_simcno_frozen_scores.json

FT14 Untried or queued training directions [open]	
Question / hypothesis	Intermediate freeze ratios, SWA, and untried architectures may beat the organizer checkpoint.
What was done	Queued K1 (fc/bn + convs.) and K2 (+ spectral_convs.3.) from sim_real_fno; SWA over freeze-bn-fc snapshots; DPOT-L, WDNO, MWT, Galerkin, DeepONet.
Data / split	none run
Result	No results. MWT/Galerkin/DeepONet are equal-or-worse than FNO in the organizer's foil benchmark (0.0210/0.0237/0.0375 vs 0.0206 rel L2).
Reading	Never executed.
Scripts / source	src/training/swa_average.py, src/training/finetune_fno_kaggle.py · docs/CAMPAIGN_REPORT.md §15.5, §15.8; results/closed_directions.json C11

B.5 Post-processing

PP01 Amplitude and mean post-processing of the FNO output (eta, lambda, mean anchoring, masked zeroing) [closed]	
Question / hypothesis	Rescaling the FNO amplitude, anchoring its time-mean to the input, or zeroing masked cells improves scores.
What was done	eta (corrector amplitude) bracketed live; lambda < 1 shrink swept; time-mean anchor alpha swept; masked-cell zeroing tested.
Data / split	SELECT/FINAL (memorised); live
Result	live: eta 0 beat eta 1.4 by +0.108 final (v8 vs v4). local: lambda shrink monotone worse (lambda=1 optimal); time-mean anchor -0.1 to -4.2 monotone (FNO mean 2.1x more accurate than input's: 0.00409 vs 0.00912; tke exactly invariant); masked-cell zeroing +0.014.
Reading	The FNO already emits the rel-optimal conditional mean; only lambda > 1 with bands fixed was untested, and TK04 later exploited that.
Scripts / source	docs/CAMPAIGN_REPORT.md §6.4, §9 headroom table, §12.7; experiments/tke_energy/tke_analysis.md §4

B.6 Blending

BL01 Output blends of FNO with DMD, CNO, Transolver, persistence [closed]	
Question / hypothesis	A blend with a model of different inductive bias reduces error.
What was done	Measured |error| correlation with the FNO on holdout; FNO+DMD weight sweep; 3,624-combination search.
Data / split	holdout (memorised)
Result	local: correlation with FNO: CNO +0.746, tmean +0.632, DMD +0.560, persistence +0.525. FNO+DMD -1.98 at 10% DMD weight, monotone. 3,624-combination search returned pure FNO (its objective omitted sps).
Reading	Every available predictor makes largely the same errors and all are much worse than the FNO.
Scripts / source	docs/CAMPAIGN_REPORT.md §6.3, §6.4, §12.6

BL02 Proportion search with sps in the objective (FNO/CNO/DMD/corrector) [inconclusive]	
Question / hypothesis	With sps included and time charged, a regime-fitted proportion of models beats the frozen FNO.
What was done	experiments/early_diagnostics/combo_symbolic.py: DMD variable sweep, then proportions on cluster-1 FIT pool, HOLDOUT touched once; experiments/early_diagnostics/combo_forms.py adds mean/fluctuation and spectral/lead/channel splices.
Data / split	cluster-1 FIT 154 windows / HOLDOUT 70 windows (memorised)
Result	local HOLDOUT: frozen rel 96.301 tke 79.886 mvpe 97.430 sps 44.573 w 72.7967; best (fno 0.95, cno 0.05, eta 0.8) 96.282/81.718/97.468/44.951 w 73.0405, delta +0.2438, 'PASS' vs +0.10 floor. Not shipped.
Reading	The pass was on a memorised holdout and the corrector component had already lost live.
Scripts / source	experiments/early_diagnostics/combo_symbolic.py, experiments/early_diagnostics/combo_forms.py · results/logs/combo_symbolic_out.txt (HOLDOUT block); experiments/early_diagnostics/combo_symbolic.py docstring

BL03 Calm-region blend of the FNO toward window mean / last frame (prediction) [live-shipped]

Question / hypothesis	In calm regions trivial window estimators beat the FNO on unseen data; the memorised holdout hid this.
What was done	pred' = Em + a(P-Em) + b(El-Em) per (lead, channel, sigma_local or d2 bin); LS fit on SELECT under proxy A/B; rel scored on FINAL.
Data / split	holdout SELECT/FINAL under proxies A (k_rel 1.534/1.665) and B
Result	proxy FINAL d_relscore (both, sl, 6 bins, fitAB): A +0.108, B +0.371; best fitA A +0.102, B +0.203. Fitted weights: lead 0 mostly last frame in calm bins, late leads toward window mean.
Reading	The FNO is worse than trivial estimators in freestream (0.0056 vs 0.0041 median error) and at lead 0.
Scripts / source	experiments/estimator_blend/base.py, experiments/estimator_blend/fit.py, experiments/estimator_blend/run_pred.py, experiments/estimator_blend/diag.py · experiments/estimator_blend/run_pred_out.txt; tools/decisions/d002_slot_plan.json evidence.v17; docs/CAMPAIGN_REPORT.md §15.9

BL04 Band-centre blend and frame-difference width table (v17 recipe) [live-shipped]

Question / hypothesis	Centring the band on a blend of FNO/window estimators (not on the prediction) and sizing widths by frame-difference rank raises T.
What was done	Centre per (lead, channel, sextile) chosen on a grid to maximise T under proxies A+B; width table per (lead group, channel, p1 decile) fitted under k_T proxies; controls separate centre from pure widening.
Data / split	holdout SELECT fit / FINAL eval under proxies A, B, AT, BT
Result	proxy FINAL, full v17 recipe (predA+bandAB+boost+p1width AT,BT) vs v8: A comp +1.401 (sps +4.311), B +2.078, AT +1.326 (sps +4.006), BT +1.696. Controls (AT): centre only (bandAB) sps +2.530; width-only g=1.2 sps +1.010; v16 boost alone comp +0.207. live v17 vs v8: sps +3.949, final +1.120.
Reading	Bands need not contain the prediction, so moving their centre toward the lower-error estimator raises coverage at the same width.
Scripts / source	experiments/estimator_blend/fit_band.py, experiments/estimator_blend/run_band.py, experiments/estimator_blend/controls.py, experiments/estimator_blend/controls2.py, experiments/estimator_blend/gsweep.py, experiments/estimator_blend/make_tables.py, experiments/estimator_blend/verify_torch.py · experiments/estimator_blend/controls2_out.txt, run_band_out.txt, controls_out.txt, gsweep_out.txt; results/live_logs/live_v17.txt

BL05 v18 studies: extra estimators, unconstrained tables, ridge widths, boost refits [inconclusive]

Question / hypothesis	More blend partners (DMD, linear extrapolation), an unconstrained LS table, a ridge width score or a refitted boost add to v17.
What was done	Four studies on the v17 tables: stage 1 estimators in pred/centre; stage 2 ridge-rank widths and boost variants; stage 3 centre refits; stage 4 boost/gain grid.
Data / split	holdout SELECT/FINAL under proxies AT, A; real memorised pool R
Result	proxy FINAL vs v17: unconstrained PEl pred AT +0.129 / A +0.141 (adding dmd/lin/all: +0.131 to +0.148); ridge-rank width nb10-40 AT +0.001 to +0.007; centre refits AT +0.020 to +0.024; best boost refits AT +0.087 to +0.124. Real memorised pool: v18 (PEl_A) comp -0.055.
Reading	The v17 recipe is near what this proxy can distinguish; the only material gain is the unconstrained table, which is negative on the memorised pool.
Scripts / source	experiments/v18_refits/study1.py, experiments/v18_refits/study2.py, experiments/v18_refits/study3.py, experiments/v18_refits/study4.py, experiments/v18_refits/lib18.py, experiments/v18_refits/ridge18.py, experiments/v18_refits/make_tables18.py, experiments/v18_refits/patch_ridge.py, experiments/v18_refits/verify18.py · experiments/v18_refits/study1_out.txt, study2_out.txt, study3_out.txt, study4_out.txt, study_ft_out.txt ('stock v18 (PEl_A)' R row)

BL06 v19: ridge forecaster L as a third blend partner [adopted]

Question / hypothesis	An honest learned forecaster adds information the FNO and window estimators lack.
What was done	pred/centre = Em + a(P-Em) + b(El-Em) + c(L-Em) per (lead, channel, sextile); constrained a,b,c>=0, a+b+c<=1 (L_con) vs unconstrained pred (L_uncon); widths refit.
Data / split	holdout SELECT/FINAL under proxies AT, BT, A, B
Result	proxy FINAL vs v17: L_con AT +0.186, BT +0.294, A +0.161, B +0.329 (sps +0.42 to +0.91); L_uncon_pred AT +0.340, A +0.331 but BT -0.071, B -0.217.
Reading	L_con is positive under all four proxies, so it was built into v19.
Scripts / source	experiments/v19_ridge_partner/blend19.py, experiments/v19_ridge_partner/lib19.py, experiments/v19_ridge_partner/verify19.py · experiments/v19_ridge_partner/blend_sxb3_out.txt; tools/decisions/d006_resume_v19.json

B.7 Evaluation methodology

EM01 Composite score weights [adopted]

Question / hypothesis	The final score is the mean of the five subscores.
What was done	Least squares on 5 complete live points; single-variable measurement from v8 vs v9; later a 176-team fit on the public leaderboard.
Data / split	live ledger; leaderboard snapshot
Result	5-point fit: rel .476, tke .086, mvpe .095, time .110, sps .243 (residual max 0.0088, cond 1107, LOO -3.74 on the DMD point). Measured w_time = 0.016508/0.173384 = 0.0952. 176-team fit: rel .453, tke .0869, mvpe .0805, time .0927, sps .2838, const 3.53 (RMSE 0.117, max resid 0.60). v8 lost 1.699 tke and final rose +0.108.
Reading	Not the mean: rel is worth ~5x tke and sps ~3x tke per point.
Scripts / source	docs/CAMPAIGN_REPORT.md §2, §9, §12.1; report/data/lb_analysis.json['weights']

EM02 Run-to-run determinism of the board (v8 vs bit-identical v9) [adopted]

Question / hypothesis	Live scores are noisy enough to hide small deltas.
What was done	Resubmitted a bit-identical prediction path with added diagnostics (v9).
Data / split	live
Result	live: rel/tke/mvpe/sps identical to 6 decimals; time 91.408779 -> 91.235395 (-0.173384); final 78.140964 -> 78.124456 (-0.016508).
Reading	Four subscores are deterministic; only time varies (~5%), so composite noise is about +/-0.017.
Scripts / source	submission_fno_diag_v9.zip · docs/CAMPAIGN_REPORT.md §12.1

EM03 Local-to-live transfer record [adopted]	
Question / hypothesis	Validated local gains transfer to the board.
What was done	Compared the local delta with the live delta for every change validated offline then shipped.
Data / split	matched/SELECT-FINAL pools (memorised); live
Result	conditional-TKE map +3.25 local -> -1.02 live (-31%); FNO swap +8.05 -> +0.561 (7%); log-POD +2.73 -> +0.03 (1%); rank modifier +6.96 -> +0.152 (2%). Incremental changes to the same model ~87% (eta 0.4 -> 1.4 tke +0.955 live).
Reading	Local deltas were measured on data the checkpoint had memorised (EM04).
Scripts / source	docs/CAMPAIGN_REPORT.md §6.5 transfer table, §3
EM04 The founding error: every local holdout is memorised by sim_real_ft [adopted]	
Question / hypothesis	Local held-out trajectories are unseen by the shipped checkpoint.
What was done	Read the checkpoint meta (iteration, best_iteration, best_val_loss) and the organizers' training setup.
Data / split	all train_real
Result	sim_real_ft was fine-tuned by the organizers on all of train_real; local 96.07/80.14/97.27 are training scores. Live 94.168/74.026/92.836 is the only honest generalisation measurement.
Reading	Explains the four transfer failures (EM03) and the failed fine-tunes at once.
Scripts / source	docs/CAMPAIGN_REPORT.md §12.3, §9
EM05 The 14-trajectory live-nearest holdout (holdout_v12_3) [adopted]	
Question / hypothesis	Holding out the 14 trajectories nearest the live fingerprint gives an honest local test.
What was done	Selected 14 nearest in the 6-D space, kept the 3 Re-ceiling (26700) trajectories in training so 14/14 are bracketed; SELECT = original quad, FINAL = 10 new; scored the frozen checkpoint.
Data / split	holdout 14 (SELECT 4 / FINAL 10), train pool 67
Result	local frozen sim_real_fno: holdout-14 (588 win) 96.296/80.715/97.154; select-4 96.088/79.915/97.379; final-10 (420 win) 96.379/81.039/97.064 vs live 94.168/74.026/92.836. Report then projected ~81.2-81.3 composite (later invalidated, EM06).
Reading	Split kept as the local pool; its absolute levels are memorised (EM06), so only deltas are usable.
Scripts / source	src/splits/build_holdout_v12_3.py, src/evaluation/score_holdout_v12_3.py, experiments/early_diagnostics/holdout_hardness_distribution.py, src/evaluation/score_one_trajectory.py · data/splits/holdout_v12_3.json (n_bracketed 14); results/holdout_v12_3_stock_scores.json; data/splits/train_pool_v12_3.json; docs/CAMPAIGN_REPORT.md §14.2-14.3
EM06 Holdout contamination: sim_real_fno scores higher on the 'holdout' than live [adopted]	
Question / hypothesis	holdout_v12_3 is unseen by sim_real_fno.
What was done	Scored the never-real-trained sim_fno and sim_cno on the same holdout as a control.
Data / split	holdout final-10
Result	local final-10: sim_fno 79.709/66.872/75.199; sim_cno 78.145/67.145/74.743; sim_real_fno 96.379/81.039/97.064 vs its own live 94.168/74.026/92.836.
Reading	The holdout was in the organizers' training set, so sim_real_fno memorised it; clean-start candidates must be judged against their own frozen starting point.
Scripts / source	src/evaluation/score_cno_ckpt.py · docs/CAMPAIGN_REPORT.md §15.2; results/holdout_v12_3_simcno_frozen_scores.json; results/closed_directions.json C13
EM07 True composite derivatives through SPS (TRUE_W) [adopted]	
Question / hypothesis	The fitted composite weights are total derivatives.
What was done	Added the dependence of SPS (via pm) on rel/tke/mvpe at the v8 operating point.
Data / split	analytic at live v8
Result	rel 0.476 -> 0.649 (+36%), tke 0.086 -> 0.131 (+53%), mvpe 0.095 -> 0.160 (+68%); rel is 4.9x tke, not 5.5x. Loss weights in src/training/finetune_fno_kaggle.py.
Reading	SPS uses the same errors as the other subscores, so every accuracy change also moves SPS.
Scripts / source	docs/CAMPAIGN_REPORT.md §14.1
EM08 Live-calibrated error-inflation proxy (proxy A/B, k_rel vs k_T) [adopted]	
Question / hypothesis	Inflating memorised-holdout residuals Y'=P+k(Y-P) until a live statistic is matched gives a local proxy that predicts live band/SPS effects.
What was done	k fitted to live rel error (k_rel SELECT 1.534 / FINAL 1.665) or to v8 live T=0.4914 (k_T 1.394 / 1.447); validated against the live DMD band bracket shape and the v6 x0.85 loss; tested prospectively on v17.
Data / split	holdout SELECT/FINAL
Result	proxy (AT, k_T) predicted v17 sps +4.006, composite +1.326; live sps +3.949, final +1.120. Under proxy A the no-shape width sweep peaks at g=1.0 (28.81/29.47/28.99 for g .74/1/1.21), same shape as the live DMD bracket (30.83/31.88/30.72). Proxies overstate band effects 1.5-1.9x; rel/mvpe off by ~0.15.
Reading	The first local proxy validated against a live result; it only models the stock FNO's error scale.
Scripts / source	experiments/interval_recipes/validate_proxy.py, experiments/interval_recipes/common.py, experiments/interval_recipes/check_scale.py, experiments/estimator_blend/gsweep.py · experiments/interval_recipes/recipes_out.txt (k values); experiments/estimator_blend/gsweep_out.txt; experiments/estimator_blend/controls2_out.txt; results/live_logs/live_v17.txt
EM10 Pre-registered decision gates for submission slots [adopted]	
Question / hypothesis	Structured probability gates over written evidence give better slot decisions than ad hoc choice.
What was done	Six structured decision records (options, evidence, pass/fail checks) written before each submission-slot decision and logged to tools/decisions.
Data / split	decision records d001-d006
Result	Chosen options: d001 tke boost; d002 wait for v17; d003 submit v17 over v16; d004 proxy search for v18; d005 fine-tune then v18; d006 build the L_con partner. The decision tool's internal scores are uncalibrated and not reported. d003's pick (v17) beat v8 live by +1.120.
Reading	Used as a cheap gate; one outcome (v17) so far checks against live.
Scripts / source	tools/jev_gate.py · tools/decisions/d001-d006*.json

Appendix C. Measurements (facts established, not hypotheses tested)

C.1 Data audit

DA01 Byte-identical duplicate trajectory 6300_0 / 7575_0 [adopted]

Question	The released train_real set may contain duplicated recordings.
How measured	Content hash of every trajectory file; duplicates reported to organizers and excluded from all pools.
Data / split	all 82 train_real files
Result	1 duplicate pair (hash c2a5a440...): 6300_0 == 7575_0. Organizers issued a data notice confirming it. 7575_0 is excluded from train_pool_v12_3 and holdout_v12_3.
Reading	A duplicated trajectory leaks into both train and validation if not removed.
Scripts / source	src/splits/make_matched_splits.py · results/eda_profile.json['duplicates']; docs/CAMPAIGN_REPORT.md §5 'Defects'

DA02 Reynolds attribute vs filename mismatch [adopted]

Question	The Re encoded in filenames is the true Reynolds number.
How measured	Compared each file's re attribute with its filename and reconstructed the true Re ladder.
Data / split	all 82 train_real files
Result	78 of 82 files have an re attribute that disagrees with the filename. True step is 1278-1279 (not 1275); true sequence 3750..26761 (18 values). The often-quoted eval 'Re 15225' is wrong.
Reading	Filenames use a rounded nominal step; the attribute gives the true value.
Scripts / source	results/eda_profile.json['re_sequence_true']; docs/CAMPAIGN_REPORT.md §5

DA03 Evaluation-set fingerprint from our own ingestion logs [adopted]

Question	Target-free per-process statistics printed by the submission identify the eval regime.
How measured	Aggregate statistics of the evaluation inputs (means, standard deviations, local noise level, frame differences, correlation time, mask fraction) were written to the participant log by diagnostic submissions and compared with train_real (Disclosure 2).
Data / split	live eval (4 processes in eda_profile.json); local reference pool
Result	A single flow condition, AoA 15, Re ~20,600-25,200. Live rel-L2 error was about 1.5x local; no uniform error or band scaling reproduced the live band factor. The logged values are withheld from this public report (Disclosure 2).
Reading	Only u_std is monotone in Re, which pins the Reynolds range; v_std/tau vary with AoA.
Scripts / source	results/eda_profile.json['live'] (4 live_procs); docs/CAMPAIGN_REPORT.md §5 table

DA04 train_sim alignment with train_real [inconclusive]

Question	train_sim could be used as extra training data after a simple rescale.
How measured	Compared grid geometry, amplitude ratios, KE-map correlation and dropout fraction between train_sim and train_real.
Data / split	train_sim vs train_real
Result	Grid matches exactly, but u_std ratio 3.59x vs v_std 10.81x (anisotropic, not a scalar), KE-map correlation 0.69, zerofrac 0.20% (sim) vs 9.91% (real). Never used for training.
Reading	The amplitude mismatch differs per component and sim has no PIV dropout, so no single rescale aligns it.
Scripts / source	docs/CAMPAIGN_REPORT.md §9 'Augmented fine-tune', §12.7, §15.5

C.2 Data analysis

EA01 Regime-matched validation splits [adopted]

Question	The v9 Re/AoA-stratified splits average over flows much less energetic than the eval set, explaining transfer failures.
How measured	Ranked all trajectories by mean abs log-distance to the live 6-stat fingerprint; dealt the 24 closest into MTRAIN/MSELECT/MFINAL.
Data / split	82 trajectories -> 66 train / 8 select / 8 final
Result	Split counts 66/8/8 (verified). v9_splits.json overstates SPS by +8.458 vs the board. Later found (EM04) to still be memorised data for sim_real_ft.
Reading	Matching the regime makes local levels comparable, but it does not make deltas transfer.
Scripts / source	src/splits/make_matched_splits.py · data/splits/matched_splits.json; src/splits/make_matched_splits.py docstring; docs/CAMPAIGN_REPORT.md §8

EA02 Generalisation map of the frozen FNO over 81 trajectories [closed]

Question	If live sits inside the per-trajectory spread the gap is ordinary generalisation; if below all, the eval flow is anomalous.
How measured	Scored frozen sim_real_ft on all 81 training trajectories (10 windows each) and placed the live scores in that distribution.
Data / split	81 train_real trajectories (all memorised)
Result	local: rel min/p5/median/max 93.46/94.17/95.97/97.91; tke 68.62/70.01/76.82/88.21; mvpe 93.34/94.41/96.45/98.74. live: rel 94.17 (p5), tke 74.03 (inside), mvpe 92.84 (below all 81). No decay with distance to live fingerprint.
Reading	Every trajectory was in the checkpoint's training set, so the map measures memorisation, not generalisation (see EM04).
Scripts / source	experiments/early_diagnostics/generalisation_map.py · results/generalisation_map.json (percentiles recomputed, match); docs/CAMPAIGN_REPORT.md §6.5

EA03 Model-side vs data-side anomaly test (pre-registered, via v9 diagnostics) [adopted]

Question	If the input decorrelation rate is normal but the model's continuity/energy are off, the live deficit is model-side and fixable.
How measured	v9 logged target-free stats dec, cont, vr live; compared with the 10 nearest training trajectories; read stated before submitting.
Data / split	live (11 visible processes) vs 10 NN train trajectories
Result	The live decorrelation rate lay inside the range of the 10 nearest training trajectories, while the model's continuation and retained energy fell just outside it. Logged values withheld (Disclosure 2).
Reading	The flow decorrelates normally while the model continues it worse and keeps less energy, so the deficit is in the model.
Scripts / source	submission_fno_diag_v9.zip · docs/CAMPAIGN_REPORT.md §12.2; v10 source comment (TARGET-FREE DIAGNOSTICS)

EA04 k-means regime clustering of the flow fingerprint [adopted]	
Question	The live flow sits in a coherent regime cluster where the FNO is weakest.
How measured	k-means (k=2,3,4) on the 6-D standardized fingerprint; per-cluster frozen-FNO scores.
Data / split	81 trajectories; centroids in cluster_centroids.npy
Result	Live lands in the same cluster at k=2,3,4. Cluster c1 (n=19, AoA 10/15/20): rel 95.00+/-0.59, tke 77.39, mvpe 95.92 (local, memorised) vs c0 97.41/70.92/98.20.
Reading	Used to define live-similar pools for later specialists and gates.
Scripts / source	src/training/cluster_specialist_kaggle.py · docs/CAMPAIGN_REPORT.md §12.5; src/training/cluster_specialist_kaggle.py docstring

EA05 Spectral decomposition of the FNO residual and per-mode linear gain [closed]	
Question	The FNO error is either a resolution (truncated-mode) problem or a linearly correctable fit problem.
How measured	Split holdout residual energy into representable vs truncated Fourier modes; fitted a convex per-mode complex gain (4,224 params, exact LS).
Data / split	5-trajectory holdout (memorised)
Result	local: 86.9% of error in representable modes, 13.1% truncated (worth +0.190 rel if fixed); W modes 0-4 hold 52.4%, H modes 0-4 60.0%. Per-mode gain: +0.007 rel holdout, +0.026 in-sample; fitted |gain| median 0.838.
Reading	The error is large-scale and not a linear function of the prediction; LS wants to shrink amplitude further.
Scripts / source	docs/CAMPAIGN_REPORT.md §12.4

EA06 Live input diagnostics (v10) vs local reference: spectrum, sigma histogram, mask, autocorrelation [closed]	
Question	Live data differs from train_real in spectrum, noise correlation or mask structure, which augmentation could close.
How measured	v10 logged radial spectrum, sigma_local histogram, mask connected components and AC lags 1-3 per process; the same stats computed on 788 local cluster-1 windows and on all AoA-15 windows.
Data / split	live 17 processes; local cluster-1 pool (19 traj, 788 windows); AoA-15 pool
Result	Within the correct regime, local and live inputs matched on every low-order statistic measured (spectrum, autocorrelation, noise level); local masks were more fragmented. Logged values withheld (Disclosure 2).
Reading	Within the correct regime local and live match on every low-order statistic, so there is no first-order gap for augmentation to close.
Scripts / source	submission_fno_step1_v10.zip, experiments/early_diagnostics/local_ref_diag.py, experiments/early_diagnostics/local_ref_aoa15.py · results/logs/v10_live.txt (per-process lines); docs/CAMPAIGN_REPORT.md §13.2, §13.3, §14.3

EA07 Candidate causes of the live anomaly, tested and refuted [closed]	
Question	The live deficit comes from data prep, cropping, covariate scale, stale normaliser, masking, stall, or a temporal gap.
How measured	One targeted offline test per hypothesis (block-mean vs ::2, every 32x64 crop, TTA rescale, per-window normalisation, mask correlation, AoA-15 residual, input-target gap).
Data / split	local trajectories vs live fingerprint
Result	All refuted: block-mean no closer; crops no match; TTA rescale explains 3%; per-window normalisation hurts by 12.5; masked-cell zeroing +0.014; stall/AoA-15 residual +0.058; temporal gap gives the opposite signature; probe masking correlation has the wrong sign.
Reading	Later explained by EM04/EM06: there was no live-specific anomaly, only memorised local baselines.
Scripts / source	docs/CAMPAIGN_REPORT.md §6.5 table, §12.7

C.3 Runtime and harness

HR01 What the harness times and how it runs + Stderr log budget vs 10,240 B cap [adopted]	
Question	Model load and first-call warm-up are charged to the time subscore.
How measured	Read per-process ingestion logs and atexit per-call timers across builds (v1, v4, v6). Also: Measured bytes/process of our logging; preflight TEST 2 checks the budget.
Data / split	live ingestion logs
Result	17 processes, 2 calls of n=5 each, ~90 scored samples; import untimed (numpy import 190 ms/process would be a 38 ms/sample floor, board charged 7.7); call 0 is a synthetic Gaussian warm-up and is not charged (v4 call0 138.6 ms, v6 cut it to 16.5 ms and board charged more). 2.9-3.6 ms/sample is charged outside predict() (v4 2.922, v6 3.607). Container: A800-80GB, torch 2.2.2, numpy 1.26.4, 256 MB zip cap, 10,240 B stderr shown. | v8 printed ~1,180-1,370 B/process, so 9-12 of 17 processes were invisible; v10 compacted to 194 B/process (3,298 B total, 17/17 visible). v11-v19 preflights warn: ~701 B x 17 = 11,917 B (over cap). v17 live stderr truncated at 20,088 B (harmless).
Reading	Loading at import moves checkpoint and CUDA init out of the timed region; the harness pays to widen fp16 bounds to float32. The harness shows only the first 10,240 B of participant stderr.
Scripts / source	src/evaluation/preflight.py · docs/CAMPAIGN_REPORT.md §4; results/logs/preflight_v10_out.txt, results/logs/preflight_v11_out.txt, experiments/estimator_blend/preflight_v17_out.txt, results/live_logs/live_v17.txt; docs/CAMPAIGN_REPORT.md §7, §13

HR02 Compilation and reduced precision in the container [closed]	
Question	torch.compile, AOT compile, TF32 or fp16 could cut inference time.
How measured	Guarded self-verifying compile/TF32 paths in the builds; v15 probed torch._export.aot_compile / torch._inductor.aot_compile live.
Data / split	live container
Result	live: torch.compile PermissionError [Errno 13] in all 17 processes even with verified-writable cache dirs; AOT FAIL with the same PermissionError (v15; only 2 of 3 AOT APIs exist in torch 2.2.2). TF32 deviation 3.09e-3 to 6.09e-3 vs 2e-4 gate, auto-reverted every time (v10 log: REV 3e-3 to 5e-3). fp16 FFT blocked: cuFFT half needs power-of-two dims, ours are (26,38,70).
Reading	The container blocks compiler-toolchain execution, and the spectral weights are numerically brittle under reduced precision.
Scripts / source	submission_fno_boosted_v6.zip, submission_fno_probe_aot_v15.zip · docs/CAMPAIGN_REPORT.md §4, §15.3; results/logs/v10_live.txt 'tf32=REV'

HR03 Host-sync contention under 17 concurrent processes [adopted]	
Question	GPU contention between processes does not affect timing (earlier measurement 1.00-1.07x).
How measured	v10 added ~15-20 .cpu()/.item() sync points for diagnostics; compared scored-call time with the single-process import benchmark.
Data / split	live, 17 processes
Result	live: fno forward 3.221 ms/sample, import bench 3.340, scored 26.010 (7.78x); ingestion 30.2 ms/sample; time_score 83.088 (v8 91.409). Earlier no-sync code: ratio 1.00-1.07x.
Reading	Each host sync is a full device sync; with ~17 processes syncing at similar moments the cost multiplies, so hot paths must do one final transfer.
Scripts / source	submission_fno_step1_v10.zip · docs/CAMPAIGN_REPORT.md §13.1; results/logs/v10_live.txt (scored=19.6-31.2 per process)
HR05 Pre-submission gate and bug register [adopted]	
Question	A fresh-subprocess import test and API/scorer checks catch crashes that local warm-interpreter scoring misses.
How measured	src/evaluation/preflight.py: fresh-subprocess import, synthetic warm-up + real call, dtypes, log budget, import-guard injection, official scorer on a 2-trajectory FINAL pair.
Data / split	preflight FINAL pair 21600_15 / 25425_15 (memorised)
Result	Every build v10-v19 with a preflight output PASSES ('SAFE TO SUBMIT', 3-4 warnings). Bugs fixed (CAMPAIGN_REPORT §7) include: no bands (~-18.3 sps), lazy POD decode in predict (-0.552 time), unguarded import (scored 0.000000), _BATCH=16 vs 5, host-side broadcasts (2.088 ms/call), mvpe without time-averaging (0.783 vs true 0.202).
Reading	The v5 zero came from an exception at import that only a clean interpreter reproduces.
Scripts / source	src/evaluation/preflight.py, src/evaluation/build_fno_zip.py, src/evaluation/build_cno_zip.py · src/evaluation/preflight.py docstring; preflight_v1{0,1,2,3}_out.txt; experiments/*/preflight_v1{7,8,8ft,9}_out.txt; docs/CAMPAIGN_REPORT.md §7

Appendix D. Data, evaluation harness, training and testing: full details

D.1 The released real data (our audit of train_real)

item	value
files	82 .h5 files (datasets aoa, re, t, u, v, x, y); 81 usable: 7575_0 is byte-identical to 6300_0 and excluded
conditions	AoA 0, 5, 10, 15, 20°; 18 Reynolds labels 3,750–26,700 in the file names
Reynolds attribute	78 of 82 files store a Re that differs from the file name; true step 1,278–1,279 (3750, 5028, 6306, …, 26761)
length and time step	282–868 frames per trajectory; Δt = 0.02 s
field size	raw 64 × 128; used at 32 × 64 via [::2, ::2]; channels u, v (p ≡ 0 for PIV)
masked vectors	exact zeros where PIV has no data (airfoil body, laser shadow): 9.5% of pixels on average (5.7–12.3% per trajectory)
normalisation	input mean (0.15496, −0.00051, 0), std (0.09681, 0.01596, 1); target mean (0.15496, −0.00052, 0), std (0.09681, 0.01596, 1)
train_sim	same grid; u_std 3.59× and v_std 10.81× the real data (anisotropic); KE-map correlation 0.69; 0.20% zeros vs 9.91%; not used for training

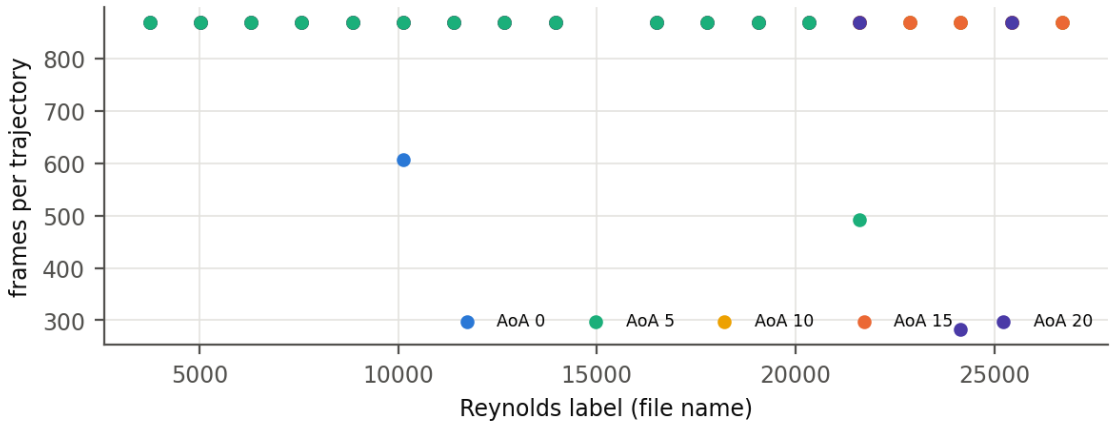


Figure 33. Length of every released real trajectory by Reynolds label and angle of attack.

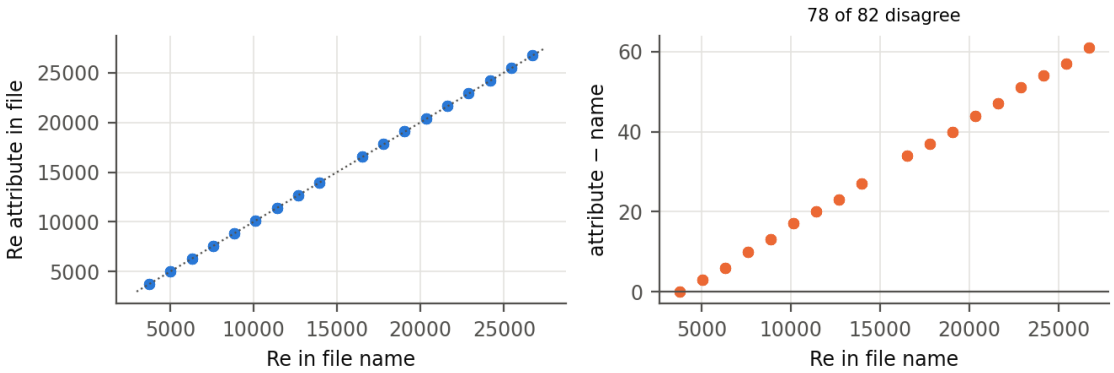


Figure 34. The Reynolds number stored in each file against the one in its name: 78 of 82 disagree, following a 1,278–1,279 step rather than 1,275.

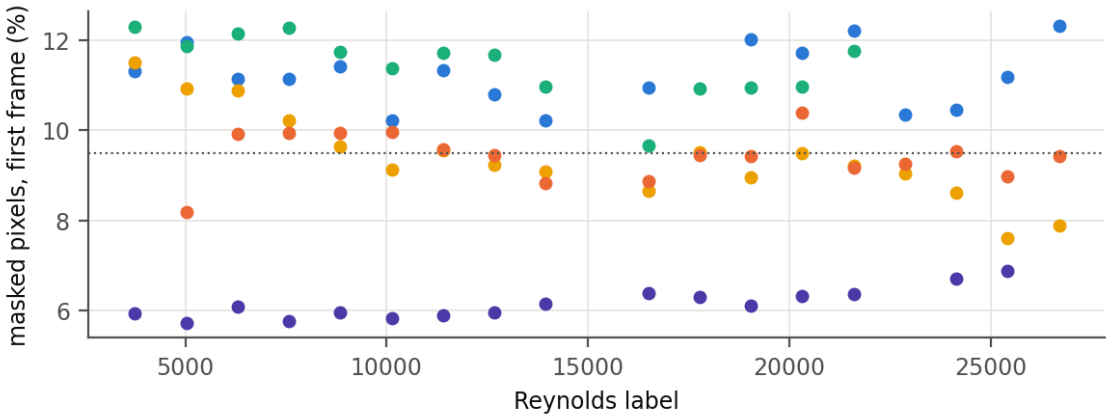


Figure 35. Fraction of masked (zero) PIV vectors in the first frame of each trajectory; dotted line: mean 9.5%.

D.2 Windows and splits

set	trajectories	windows (stride)	use
training pool	67	27,193 (stride 2, Kaggle)	fine-tuning; ridge forecaster (13,629 windows, stride 4)
SELECT	4: 21600_15, 22875_15, 24150_15, 25425_15	168 (stride 20) · 1,660 (stride 2)	epoch choice, table fitting; live-regime check
FINAL	10 (AoA 10/15/20, Re 17.8–25.4k)	420 (stride 20) · 4,150 (stride 2)	touched once per experiment
preflight pair	21600_15, 25425_15 (part of SELECT)	–	packaging gate score
random split (K9)	10125_5, 12675_10, 7575_15, 3750_20	168 (stride 20)	fingerprint-free refit

Each window is 20 input frames followed by 20 target frames. Only pixels whose target is non-zero are scored.

D.3 The evaluation harness (measured from our submissions)

item	value
hardware / software	NVIDIA A800-SXM4-80GB; torch 2.2.2; numpy 1.26.4
process layout	17 processes, each: import, one untimed synthetic warm-up call, one timed call of n = 5 windows (~90 scored)
what is timed	only predict() of the timed call; import, checkpoint load and warm-up are free; ~2.9–3.6 ms/sample charged outside predict()
unavailable	torch.compile and AOT compilation (PermissionError); TF32 (deviation 3–6×10^{-3}, auto-reverted); fp16 FFT (needs power-of-two sizes; ours 26/38/70)
limits	256 MB zip; 5 min per run; 10,240 B of stderr shown; bounds must be finite with lower ≤ upper or every subscore is zero
contention	host synchronisations inside predict() cost up to 7.8× under the 17 concurrent processes

D.4 The live evaluation set

A single flow condition, AoA 15°, Re ≈ 20,600–25,200, with masked cells exactly zero. Its low-order input statistics match local trajectories of the same regime. The logged values and figures behind this statement are withheld from this public report until the organisers rule on Disclosure 2.

D.5 Training (Kaggle Tesla T4, 14.56 GB)

setting	value
FNO fine-tune (Run 1/2, Exp. 1)	start sim_fno.pth; AdamW lr 1e-4, weight decay 1e-5, cosine decay; batch 4 × accumulation 4; 1,200 steps/epoch (4,800 windows); 12 epochs; input noise σ 0.02; fp32; ~222 ms/batch, ~300 s/epoch; best epoch by SELECT sum
loss	weighted rel-L2 / tke / mvpe errors, weights 0.734 / 0.106 / 0.160 (score sensitivity × composite weight at the start errors)
Run 2 extra	residual-to-last-frame gate (20 leads × 2 channels), initialised at 0, lr 1e-2
official-FNO fine-tunes	lr 2e-5 (Exp. 2/3), stride 20 or 2, loss weights varied (FT01/FT02), amplitude jitter 0.70 (FT02)
output-layer-only	9,667 of 50,357,955 parameters trainable (fc0, fc1, fc2, BatchNorm affine); spectral augmentation; 30 epochs
CNO fine-tune	batch 16 × accumulation 8 with AMP (batch 64/32 ran out of memory); ~5.8 s/batch; never completed
ridge forecaster (v19)	last 10 frames × 5×5 u,v neighbourhood + window mean (549 features); λ 1e-4; per σ-sextile × 4×4 block

setting	value
post-processing tables	fitted on SELECT in minutes on a CPU; blend by constrained least squares; widths by maximising exp(−2h/σ)·F(h)

D.6 Testing and validation

check	what it does
official scorer	scoring.py from starting kit v9, run locally on cached windows
honest hold-out	SELECT/FINAL for models that never saw them (sim_fno start); FINAL touched once
proxies (official FNO)	A: $Y' = P + k(Y - P)$, k_rel 1.534 (SELECT) / 1.665 (FINAL); AT: k_T 1.394 / 1.447; B/BT degrade P instead
src/evaluation/preflight.py	zip size and layout, fresh-process import, warm-up + real call, dtypes, stderr budget, failure-injection fallback, scorer acceptance
independence test	every window's prediction and bounds identical ($< 10^{-6}$) whether predicted alone, in batches, shuffled, padded or duplicated
torch vs numpy	the shipped submission.py reproduces the numpy evaluation to $\leq 1.5\times10^{-7}$ given the same bins
live board	deterministic for rel/tke/mvpe/SPS; runtime varies ±0.017 final